

Other logic gates

Other logic operations

- For n variables, 2^{2^n} boolean functions are possible

Table 2.7

Truth Tables for the 16 Functions of Two Binary Variables

[illegible]

Boolean Functions

- Two constant functions: $F_0 = 0; F_{15} = 1$
- AND/OR: $F_1 = xy; F_7 = x + y$
- Transfer: $F_3 = x; F_5 = y$
- Complement: $F_{10} = y'; F_{12} = x'$

Other logic operations

- Inhibition:
 - $F_2 = xy'$. x is true and y is not true
 - $F_4 = x'y$. y is true and x is not true
- Implication:
 - $F_{11} : x + y' = x \subset y$. When y is true, x must be true.
 - $F_{13} : x' + y = x \supset y$. When x is true, y must be true.
- Used by logicians. *rain* $\subset$ *wet*. Not much in digital logic as they are **not commutative**

Other logic operations

- Can write each one using ANDs, ORs, and NOTs
- Four remaining functions:
 - NAND: Not of AND
 - NOR: Not of OR
 - XOR: Are x and y not equal?
 - Equivalence: Are x and y equal? XNOR.

Boolean Functions	Operator Symbol	Name	Comments
$F_0 = 0$		Null	Binary constant 0
$F_1 = xy$	$x \cdot y$	AND	x and y
$F_2 = xy'$	x/y	Inhibition	x, but not y
$F_3 = x$		Transfer	x
$F_4 = x'y$	y/x	Inhibition	y, but not x
$F_5 = y$		Transfer	y
$F_6 = xy' + x'y$	$x \oplus y$	Exclusive-OR	x or y, but not both
$F_7 = x + y$	$x + y$	OR	x or y
$F_8 = (x + y)'$	$x \downarrow y$	NOR	Not-OR
$F_9 = xy + x'y'$	$(x \oplus y)'$	Equivalence	x equals y
$F_{10} = y'$	y'	Complement	Not y
$F_{11} = x + y'$	$x \subset y$	Implication	If y, then x
$F_{12} = x'$	x'	Complement	Not x
$F_{13} = x' + y$	$x \supset y$	Implication	If x, then y
$F_{14} = (xy)'$	$x \uparrow y$	NAND	Not-AND
$F_{15} = 1$		Identity	Binary constant 1

Logic gates

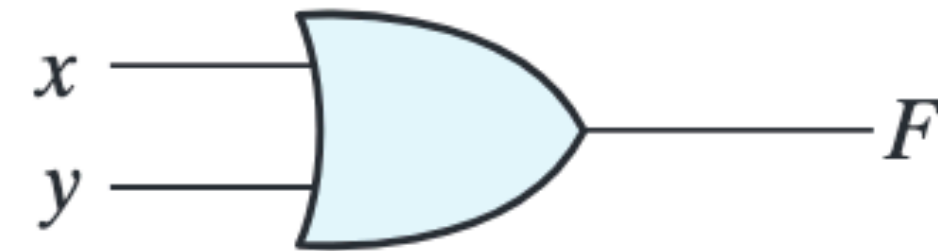
AND



$$F = x \cdot y$$

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

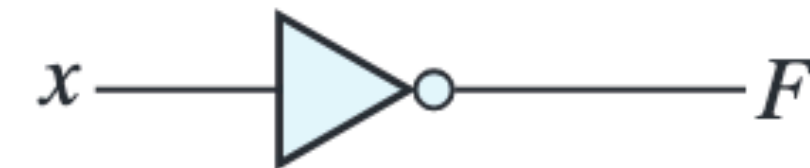
OR



$$F = x + y$$

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

Inverter



$$F = x'$$

x	F
0	1
1	0

Buffer



$$F = x$$

x	F
0	0
1	1

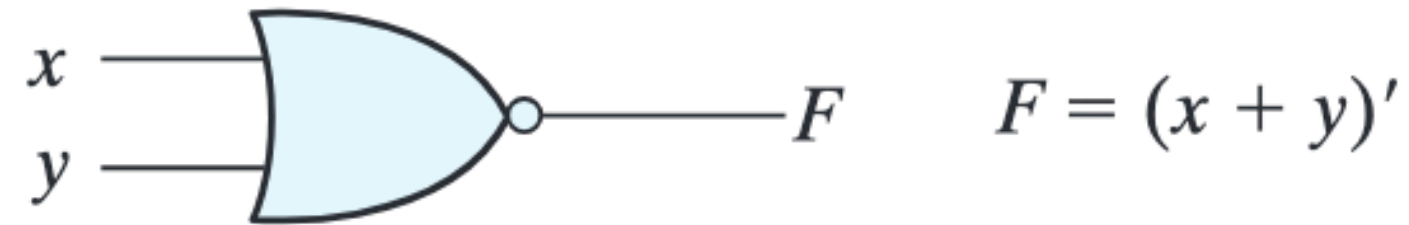
Other logic gates

- XOR adds another line to OR
- NAND, NOR, XNOR add a bubble
- All of them are commutative

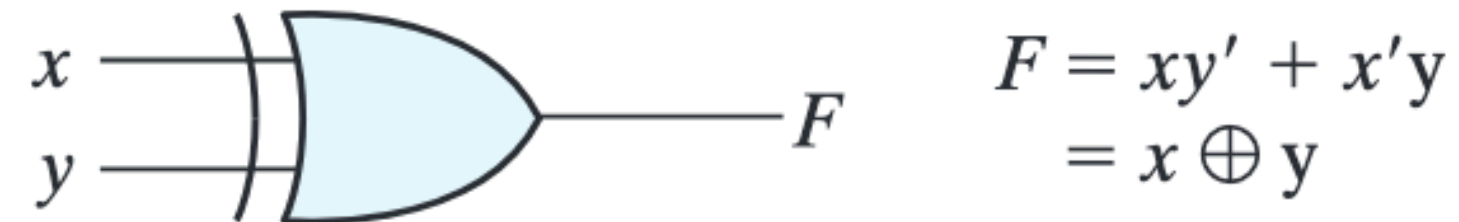
NAND



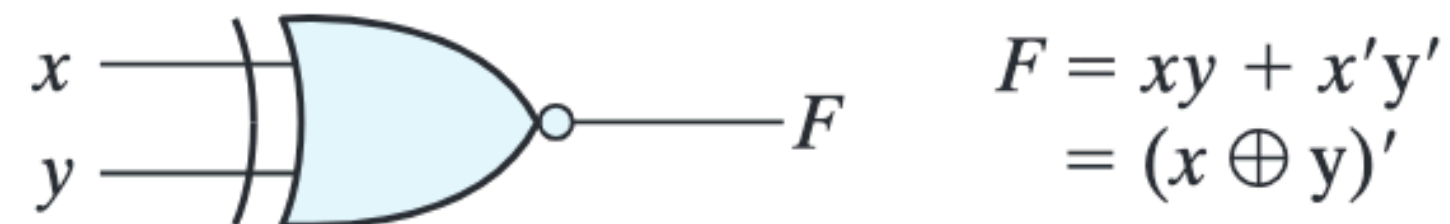
NOR



Exclusive-OR
(XOR)

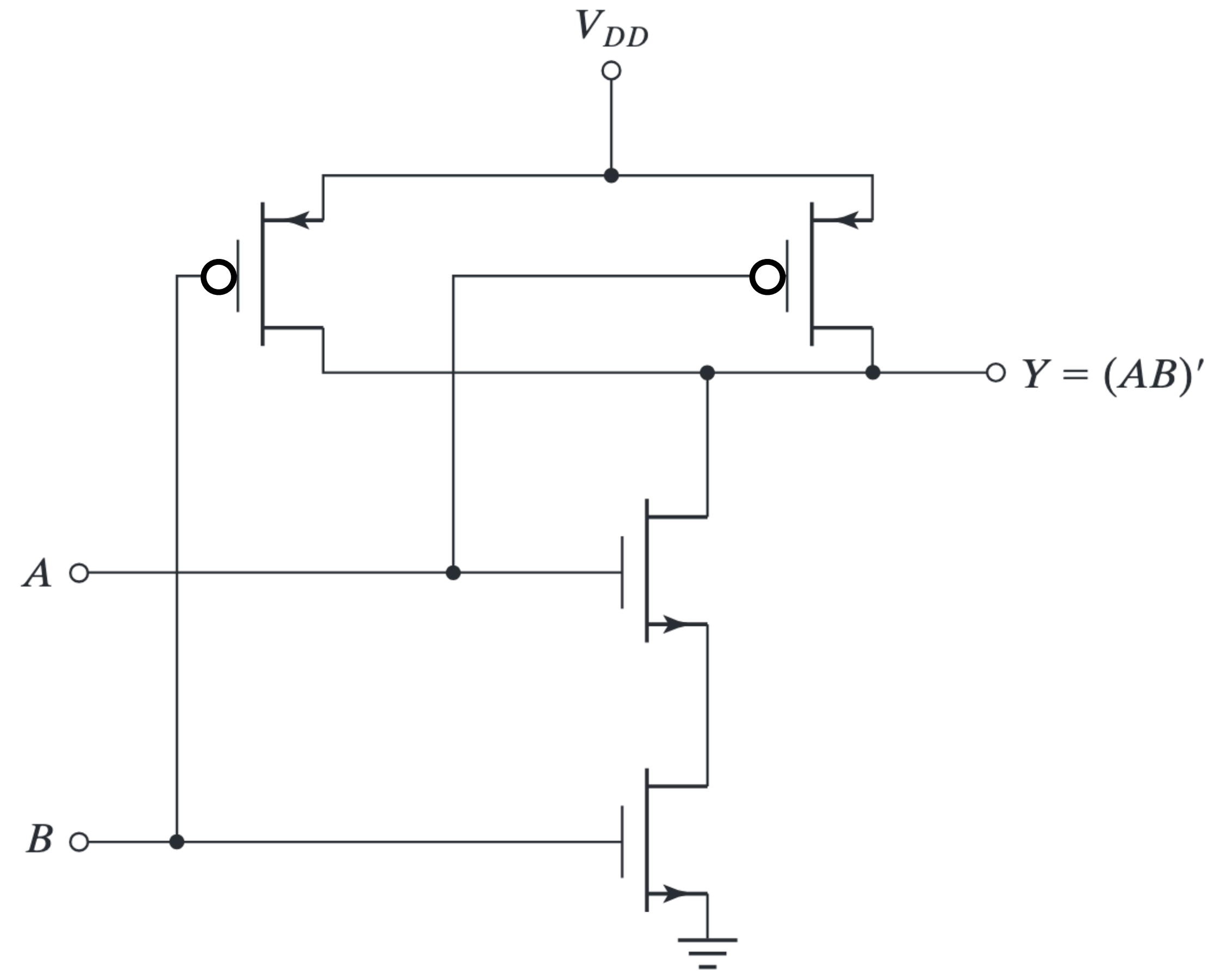


Exclusive-NOR
or
equivalence



NAND gate is easy to construct from transistors

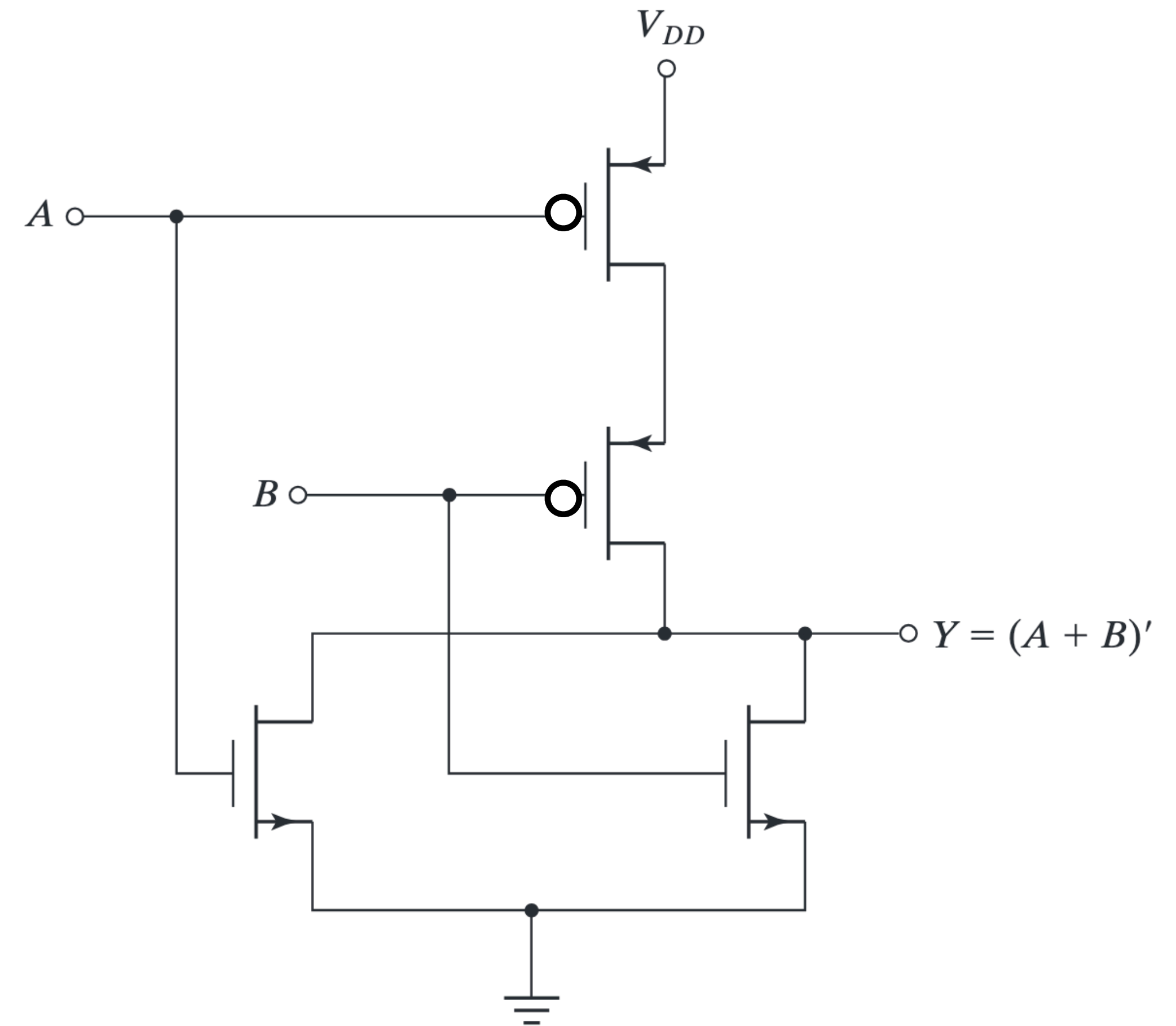
- When $A = GND$ and $B = GND$,
 $Y = V_{DD}$
- When $A = GND, B = V_{DD}$.
 $Y = V_{DD}$



(b) NAND gate

NOR gate is easy to construct from transistors

- When $A = GND$ and $B = GND$,
 $Y = V_{DD}$
- When $A = GND, B = V_{DD}$.
 $Y = GND$
- AND, OR, and XOR each require 6 transistors, instead of 4 for NOR and NAND



(c) NOR gate

Extension to multiple inputs

AND and OR

- AND and OR could be easily extended as they are both associative and commutative
- $x + y = y + x;$
 $x + (y + z) = (x + y) + z$
- $x \cdot y = y \cdot x;$
 $x \cdot (y \cdot z) = (x \cdot y) \cdot z$

Extension to multiple inputs

XOR and XNOR

- XOR and XNOR are associative and commutative

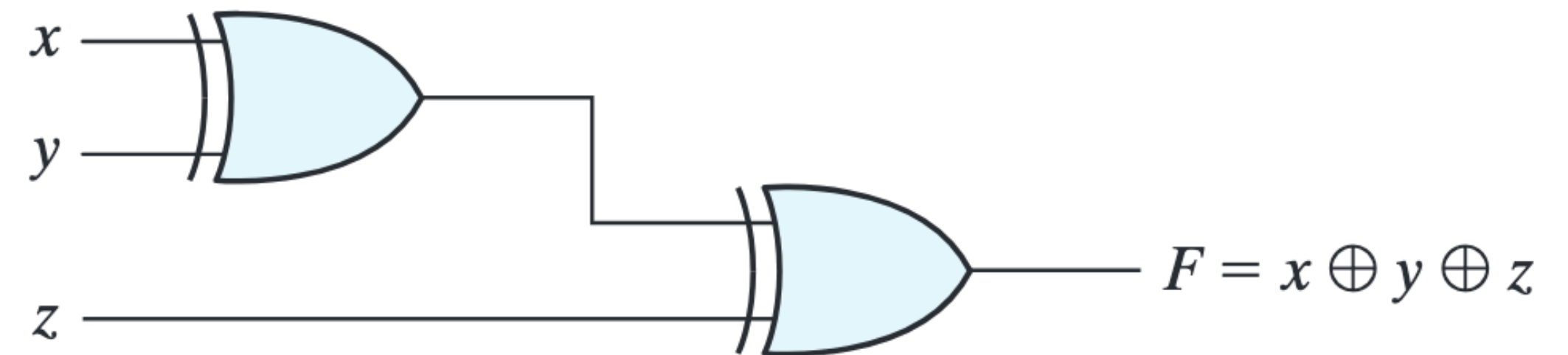
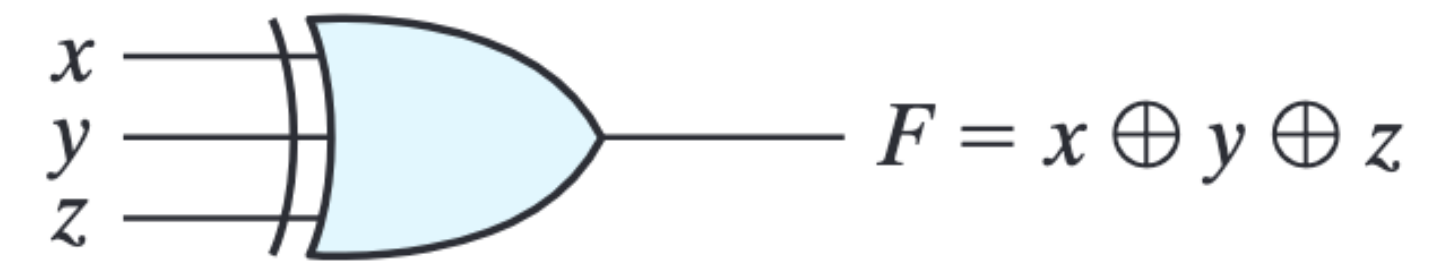
- $(xy' + x'y)'z + (xy' + x'y)z'$

- $(x'y' + xy)z + (xy' + x'y)z'$

- $x'y'z + xyz + xy'z' + x'yz'$

- $x'(y'z + yz') + x(yz + y'z')$

- $x'(y'z + yz') + x(y'z + yz')'$



Extension to multiple inputs

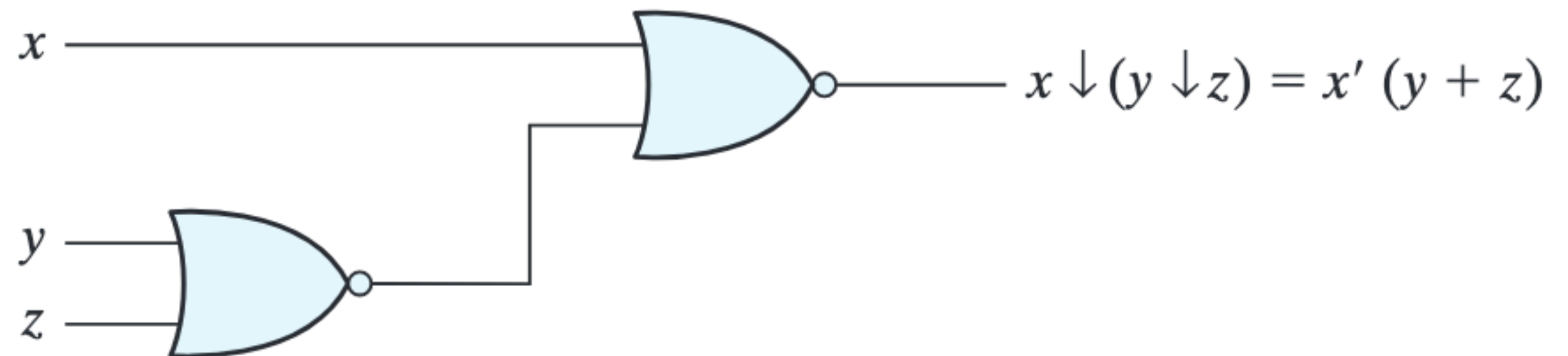
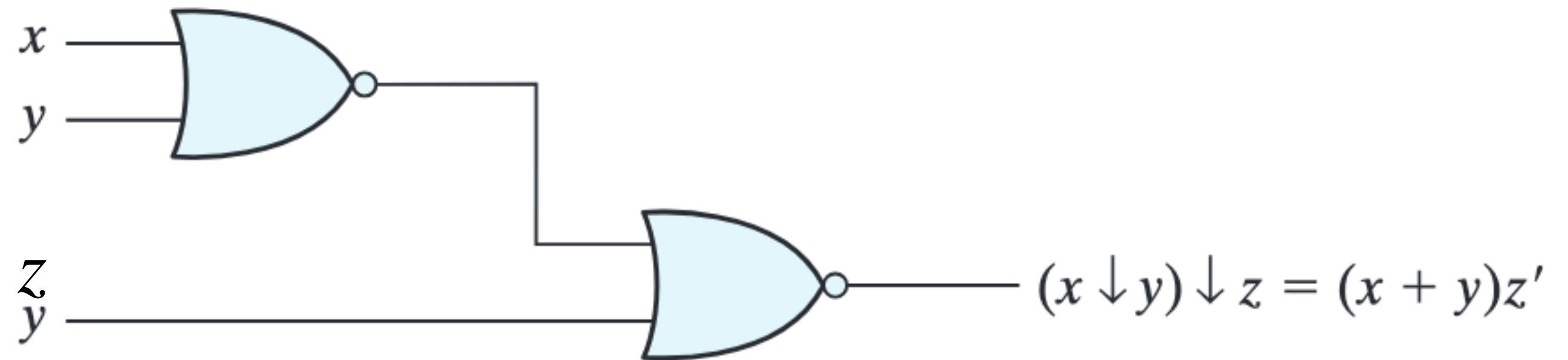
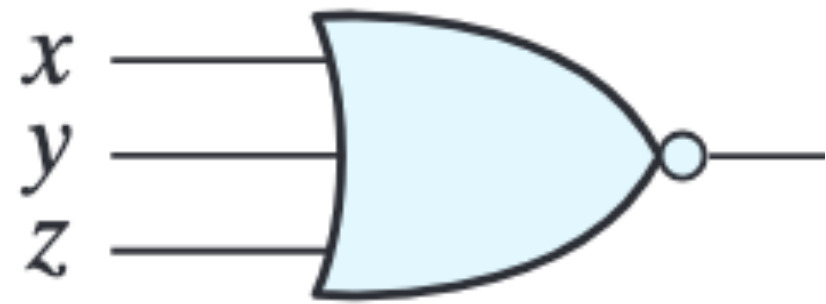
Difficulty with NOR and NAND

- NOR
 - $[(x + y)' + z]' = (x + y)z' = xz' + yz'$
 - $[x + (y + z)']' = x'(y + z)' = x'y + x'z$
- NAND
 - $[(xy)'z]' = xy + z'$
 - $[x(yz)']' = x' + yz$

Extension to multiple inputs

Difficulty with NOR (2)

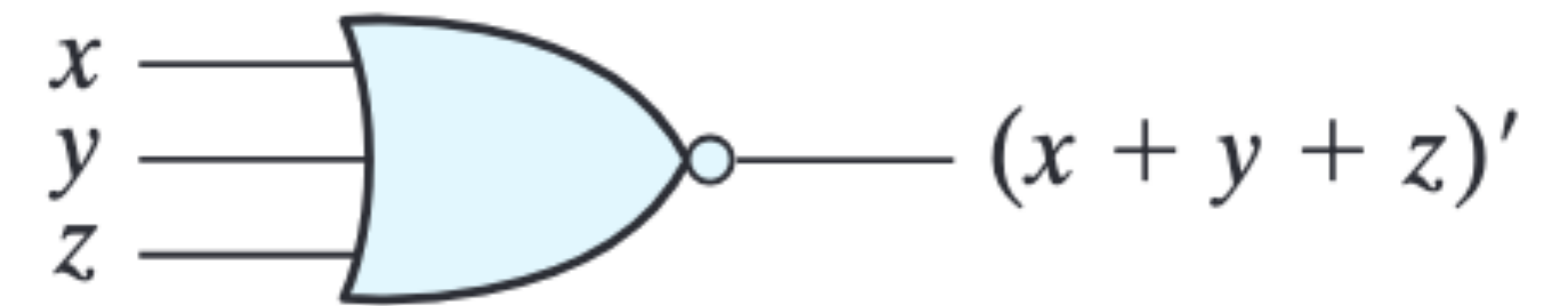
- Which circuit is this equivalent to?



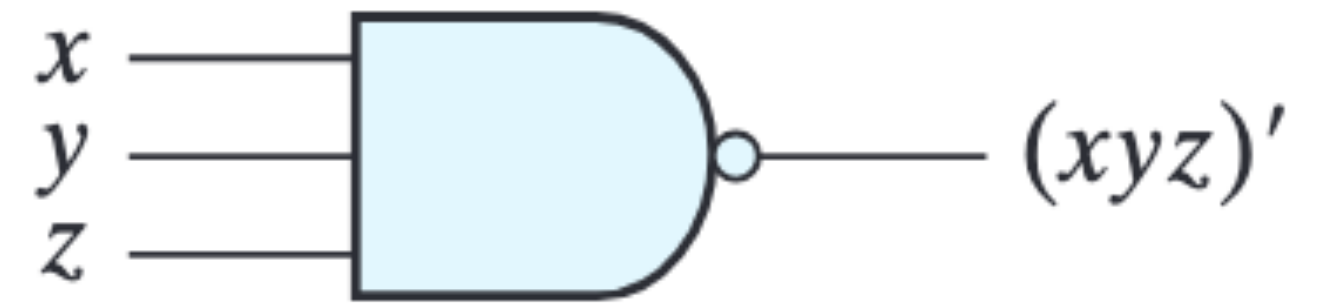
Extension to multiple inputs

NOR and NAND

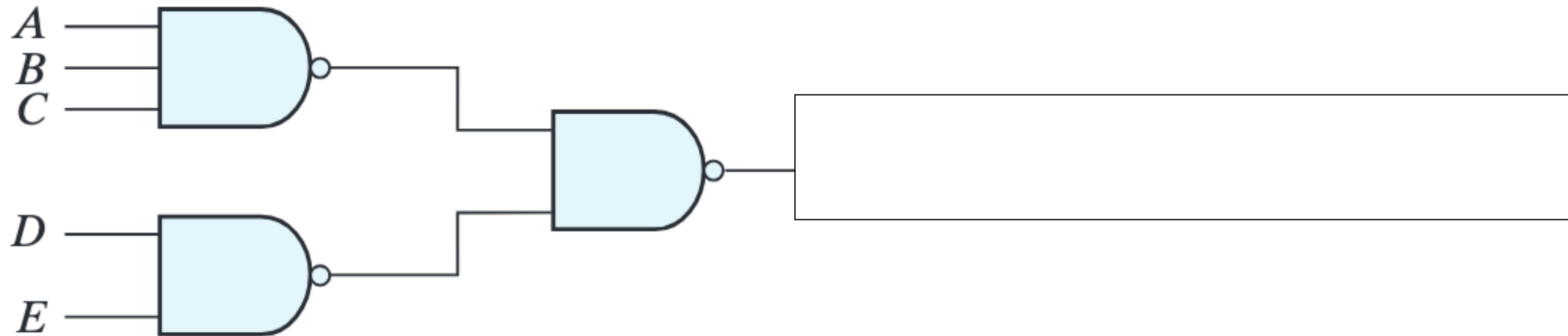
- Treat NOR as “complemented OR” since OR is associative



(a) 3-input NOR gate

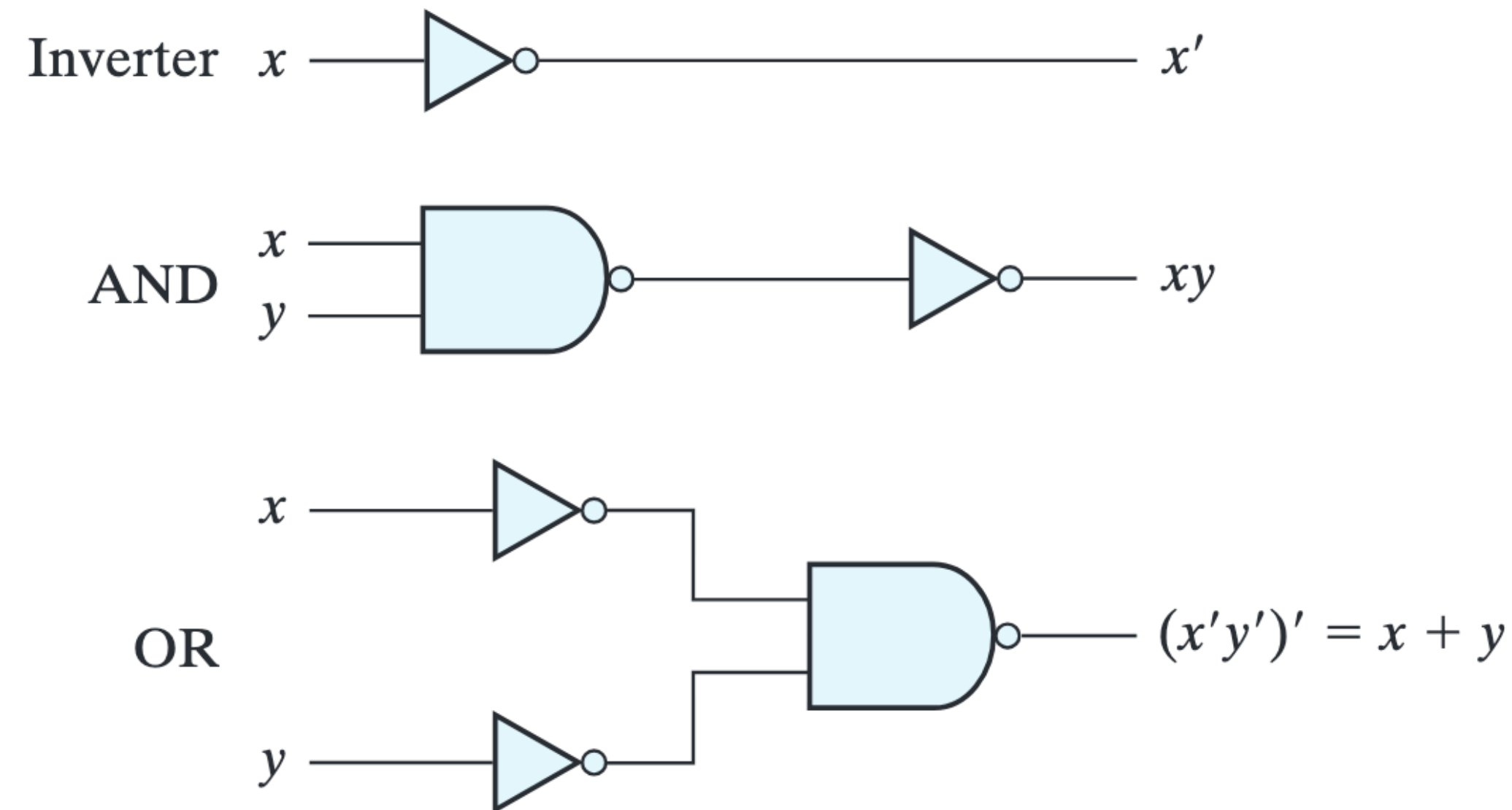


(b) 3-input NAND gate



NAND gate is *universal gate*

- Any function can be implemented with AND, OR, NOT gates (sum-of-products/product-of-sums)



Convert functions to NAND circuit

- Convert AND-OR diagrams (sum-of-products) to NAND diagrams

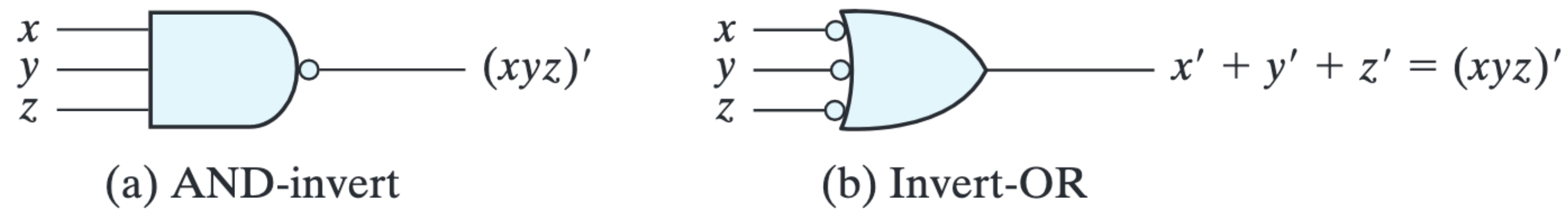


FIGURE 3.17

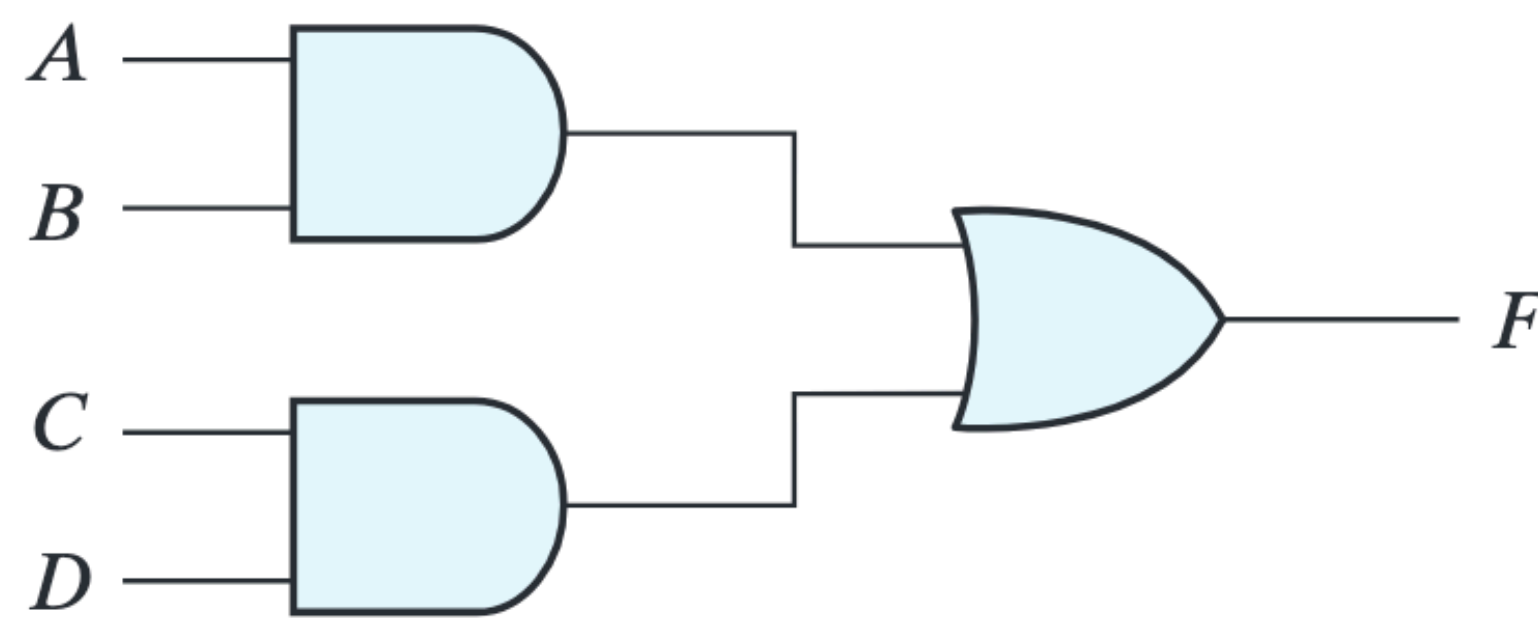
Two graphic symbols for a three-input NAND gate

Convert sum-of-product functions to NAND circuit

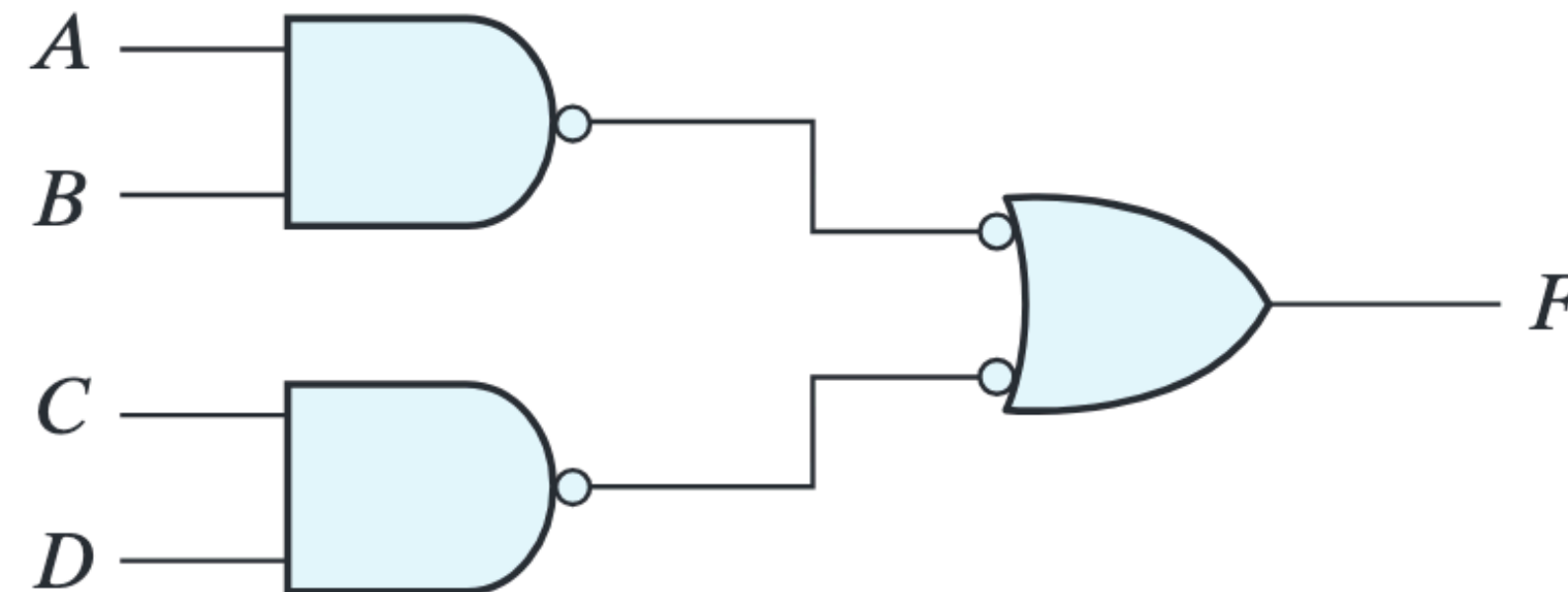
$$F = AB + CD$$

1. Replace AND gates with AND-invert gate. OR with Invert-OR gate
2. Two bubbles represent double-complement: equal to no-complement
3. Convert Invert-OR to AND-invert

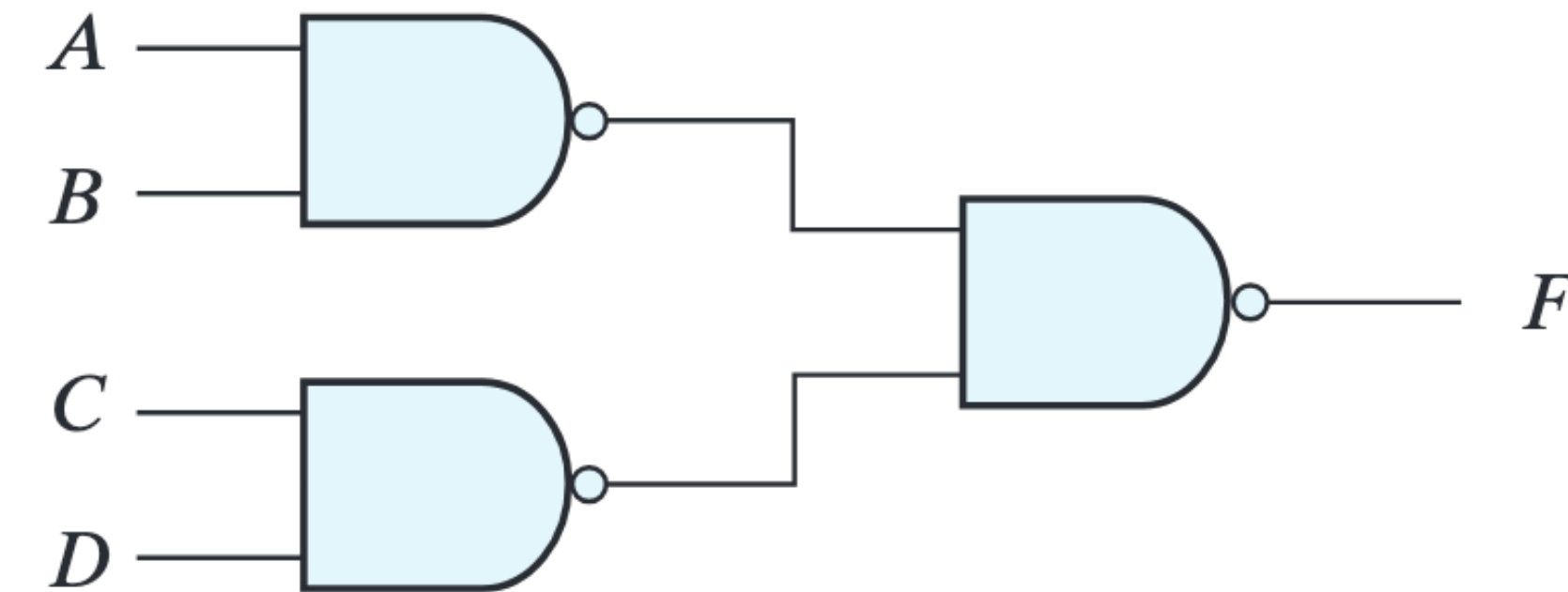
$$F = ((AB)'(CD)')' = AB + CD$$



(a)



(b)

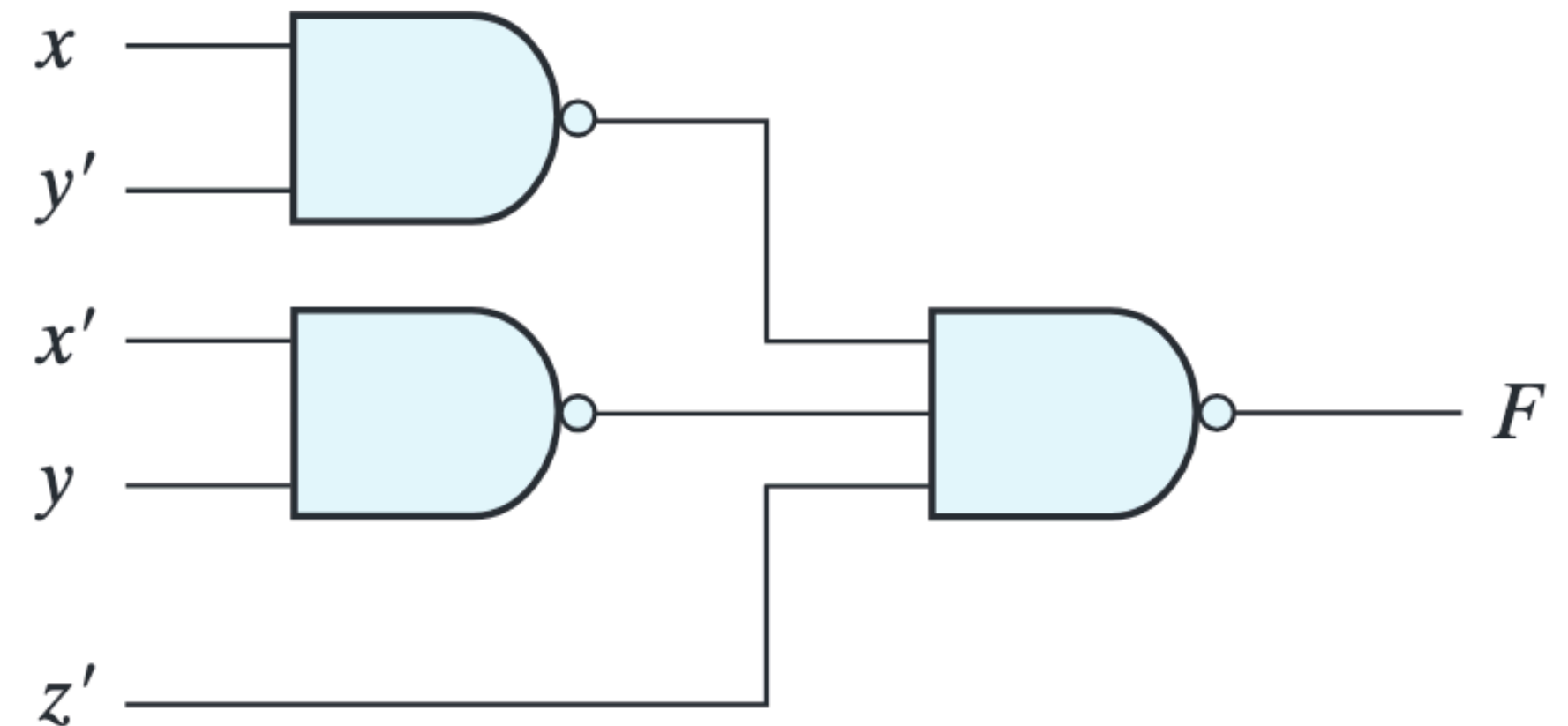
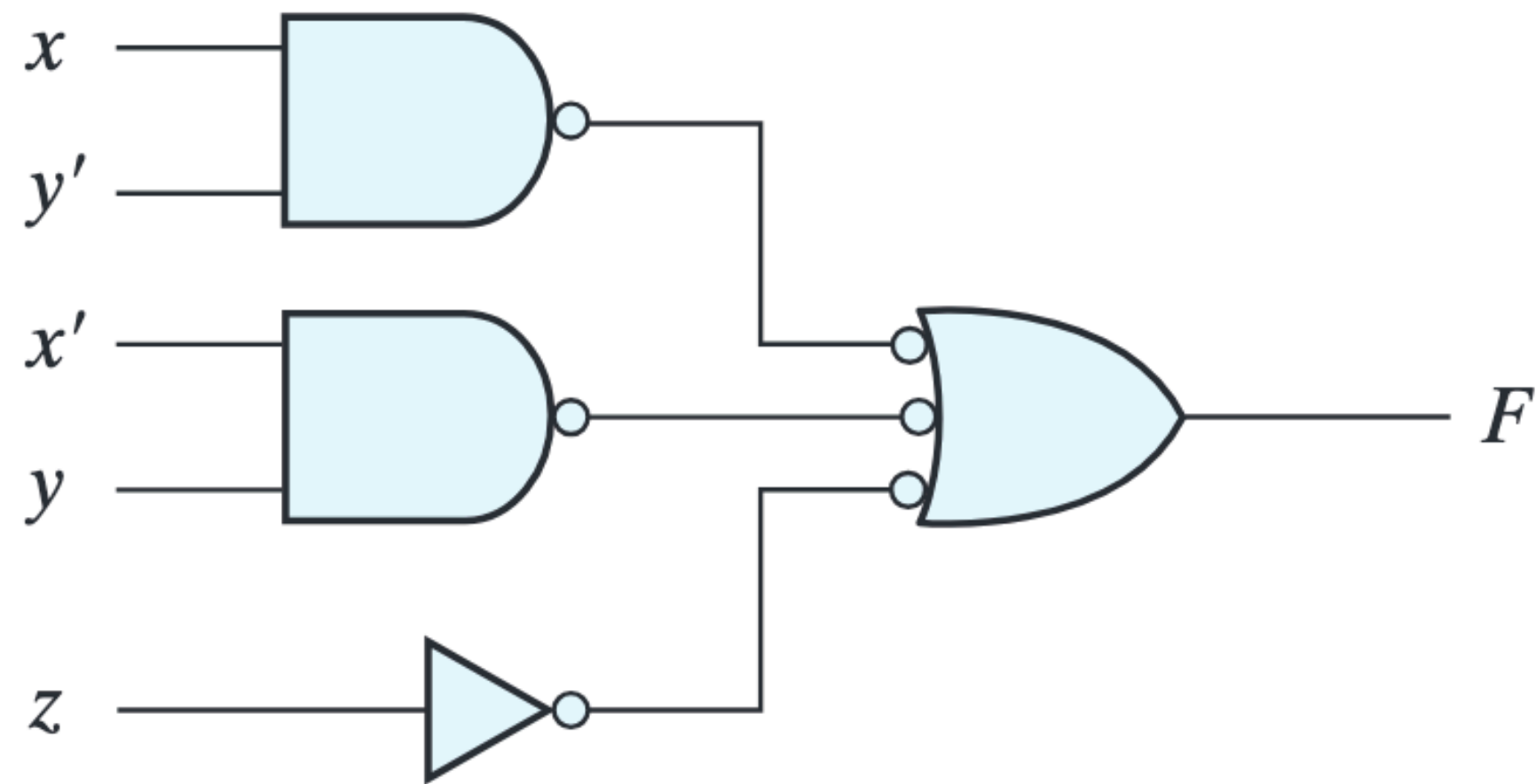
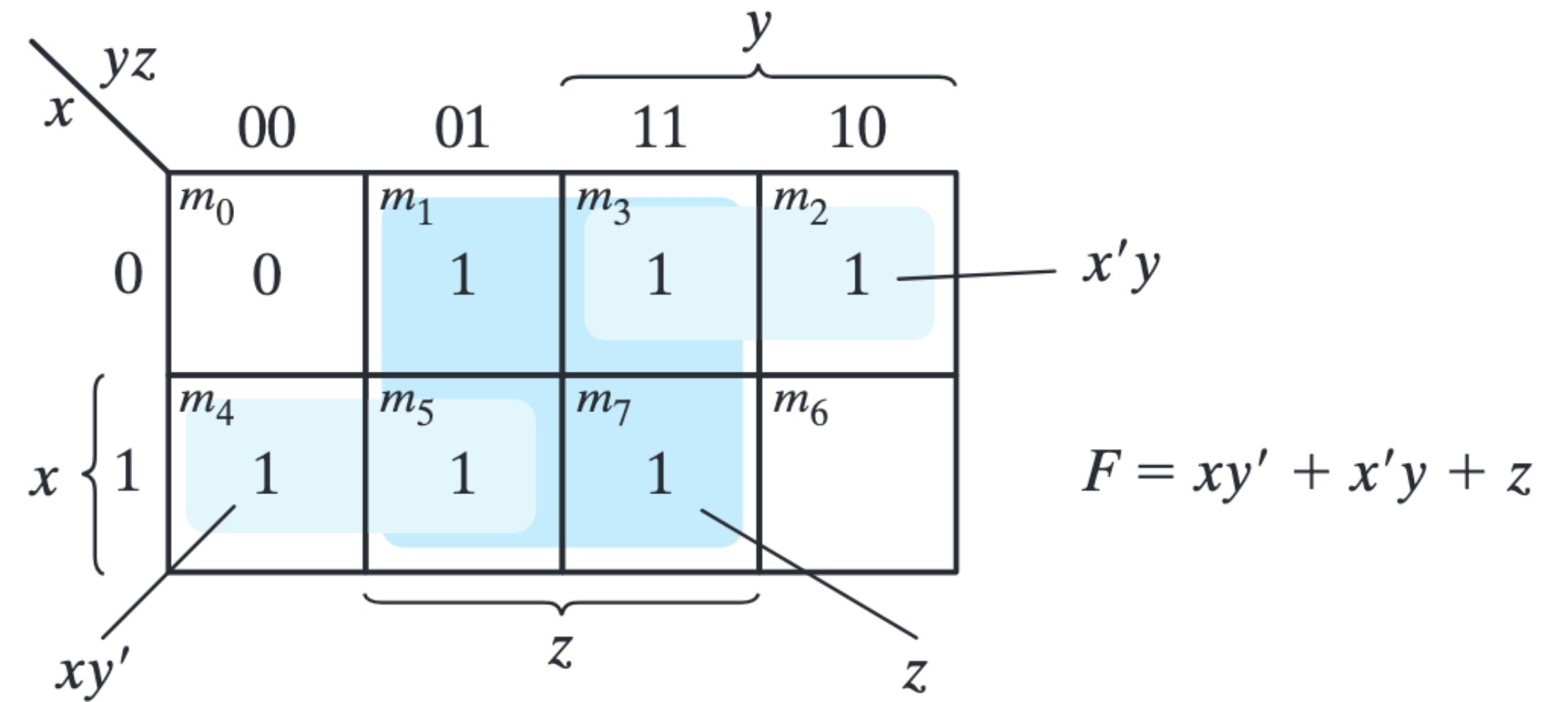


(c)

Convert canonical functions to NAND circuit

$$F(x, y, z) = \Sigma(1,2,3,4,5,7)$$

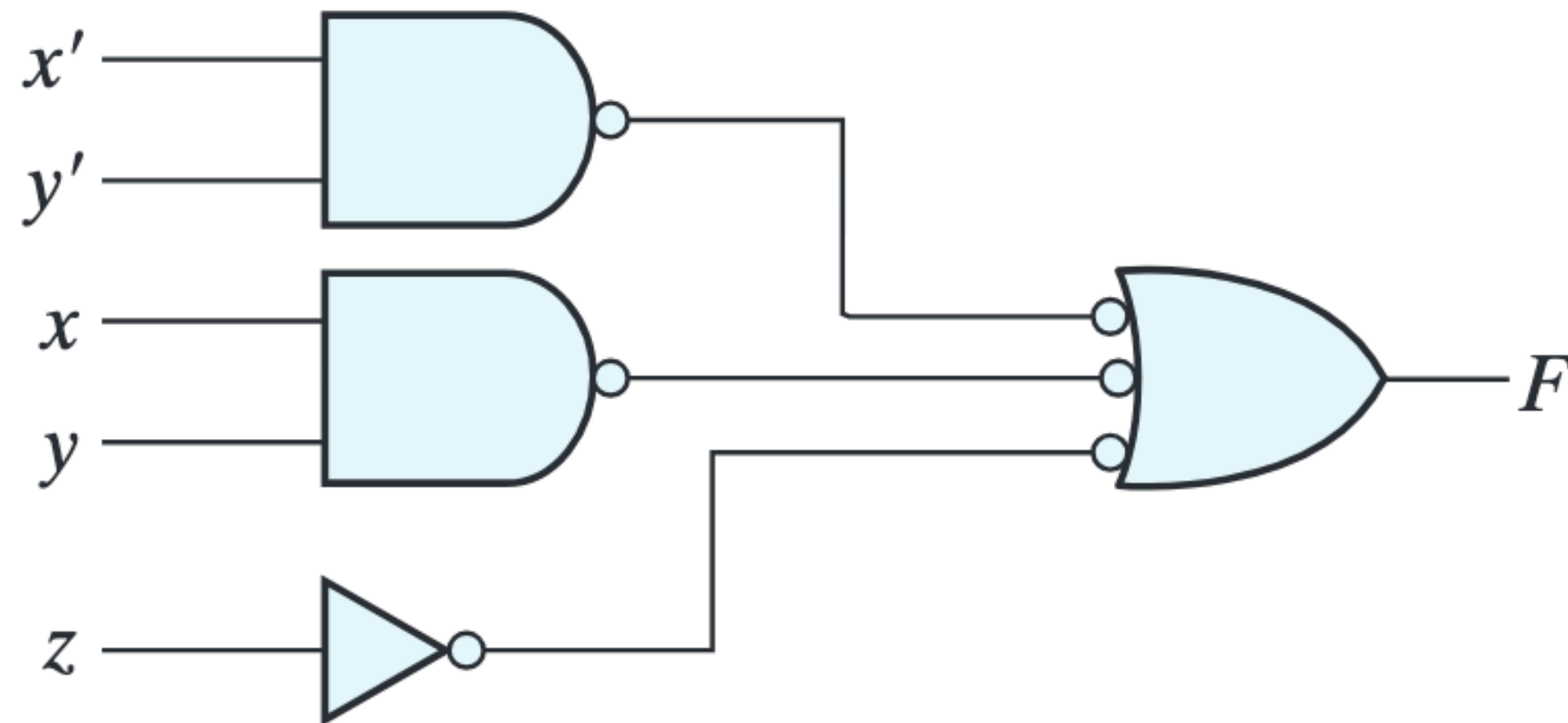
- Notice that we need to invert z



Convert canonical functions to NAND circuit

$$F(x, y, z) = \Sigma(0,1,3,5,6,7)$$

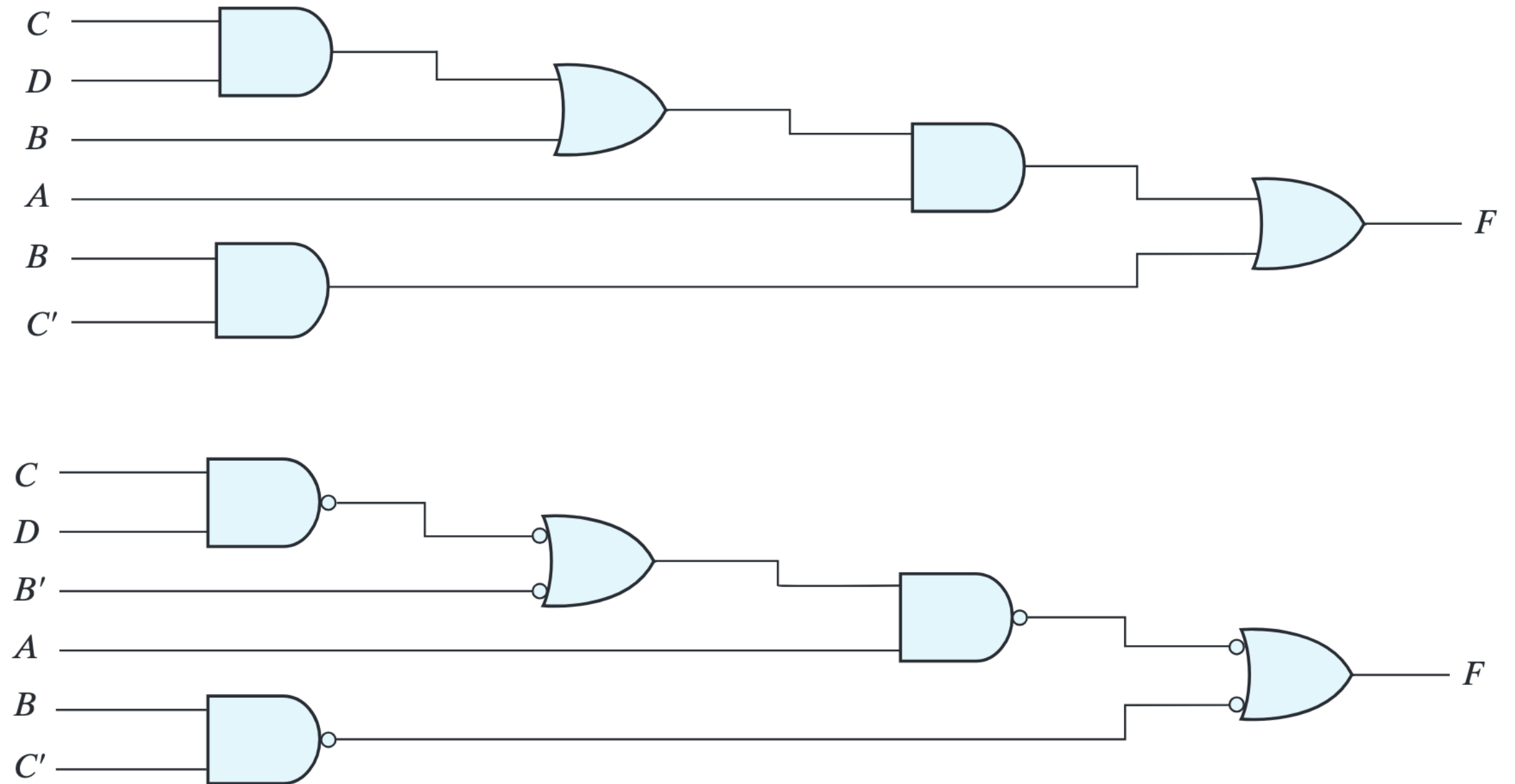
- $F(x, y, z) = x'y' + xy + z$



Multilevel NAND circuits

$$F = A(CD+B) + BC'$$

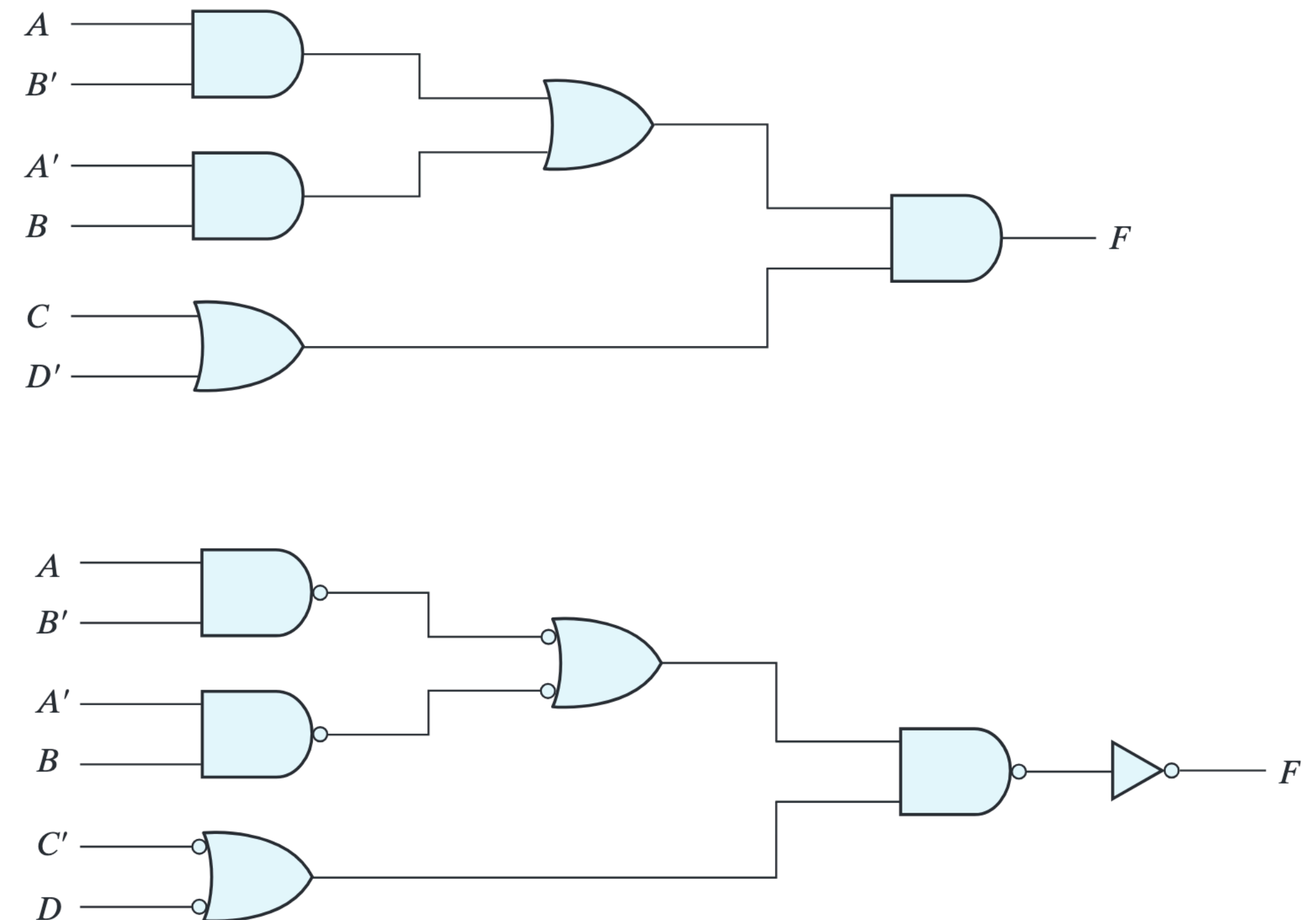
- Change AND to AND-invert, OR to invert-OR
- For every bubble that is not compensated, introduce inverter (e.g., $B \rightarrow B'$)



Multilevel NAND circuits

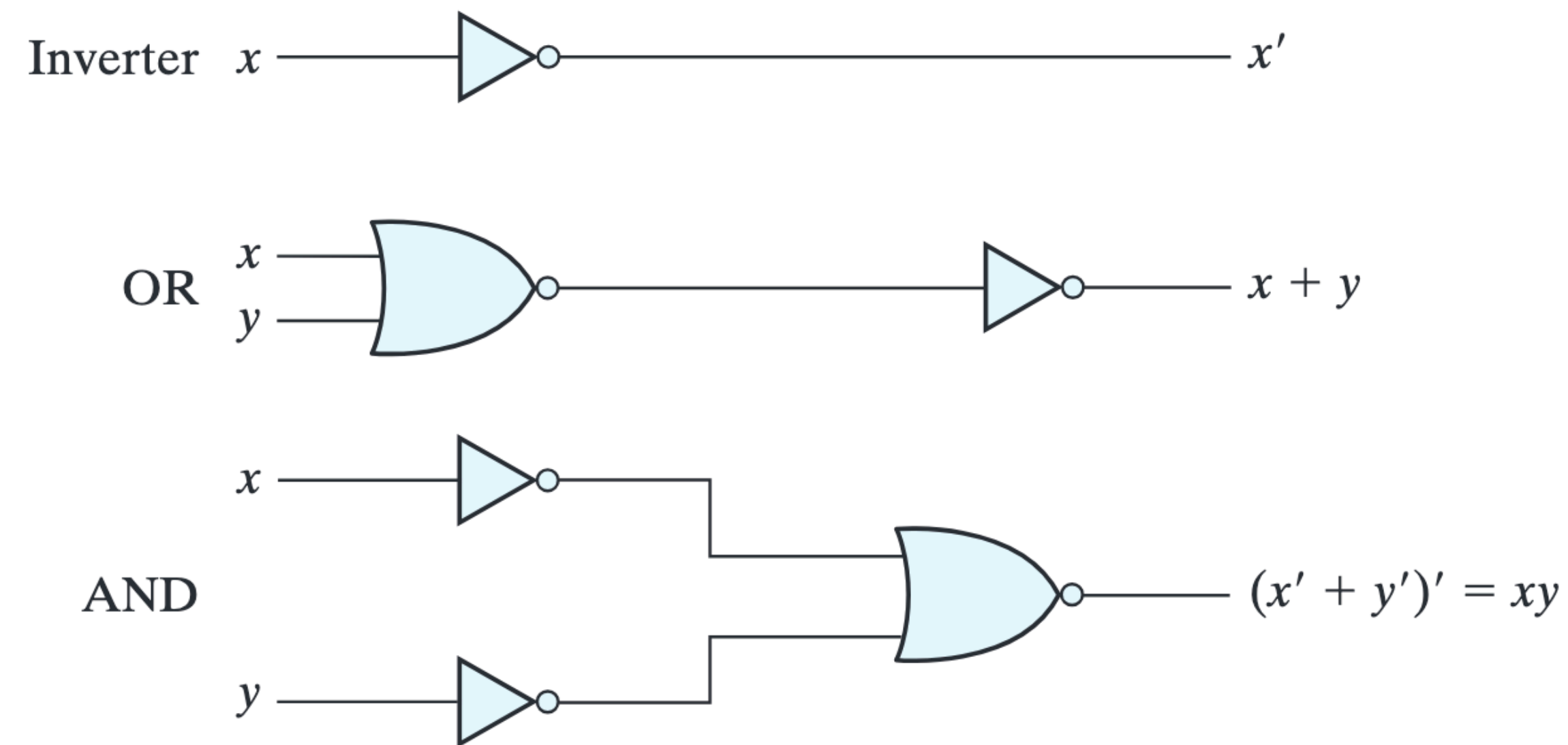
$$F = (AB' + A'B)(C + D')$$

- Change AND to AND-invert, OR to invert-OR
- For every bubble that is not compensated, introduce inverter (e.g., $C \rightarrow C'$, $D' \rightarrow D$, before F)



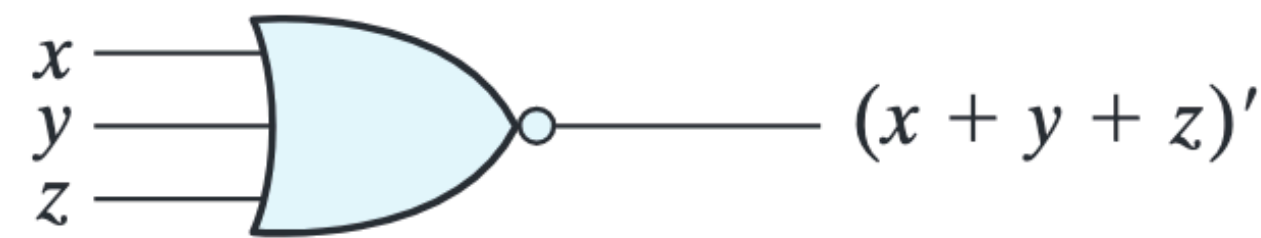
NOR gate is a *universal gate*

- Any function can be implemented with AND, OR, NOT gates (sum-of-products/product-of-sums)

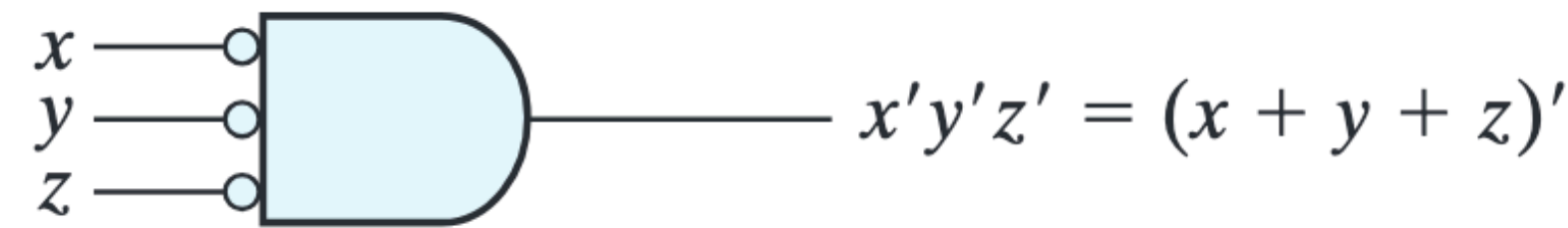


Convert functions to NOR circuit

- Convert OR-AND diagrams (product-of-sums) to NOR diagrams



(a) OR-invert



(b) Invert-AND

FIGURE 3.23

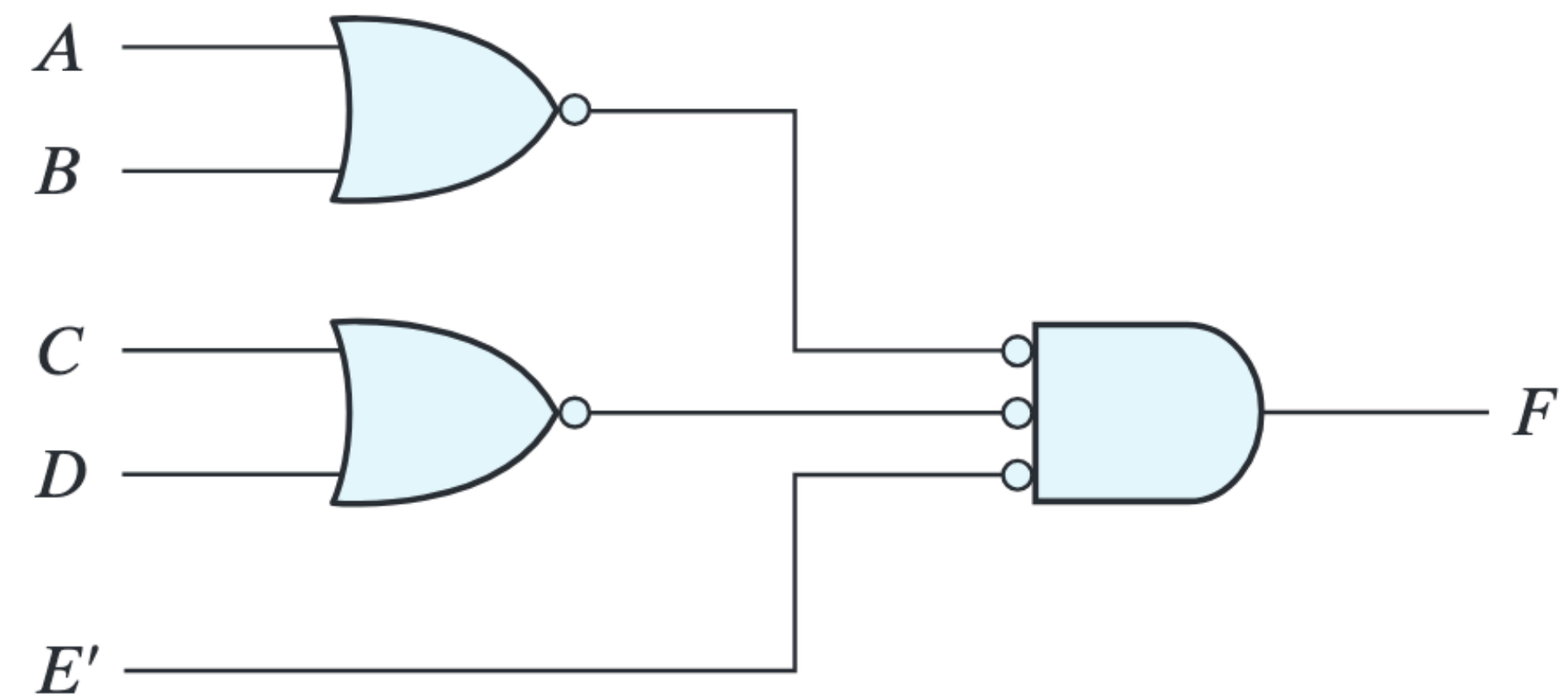
Two graphic symbols for the NOR gate

Convert product-of-sums functions to NOR circuit

$$F = (A + B)(C + D)E$$

1. Replace OR gates with OR-invert gate. AND with Invert-AND gate
2. Two bubbles represent double-complement: equal to no-complement
3. Convert Invert-AND to OR-invert.

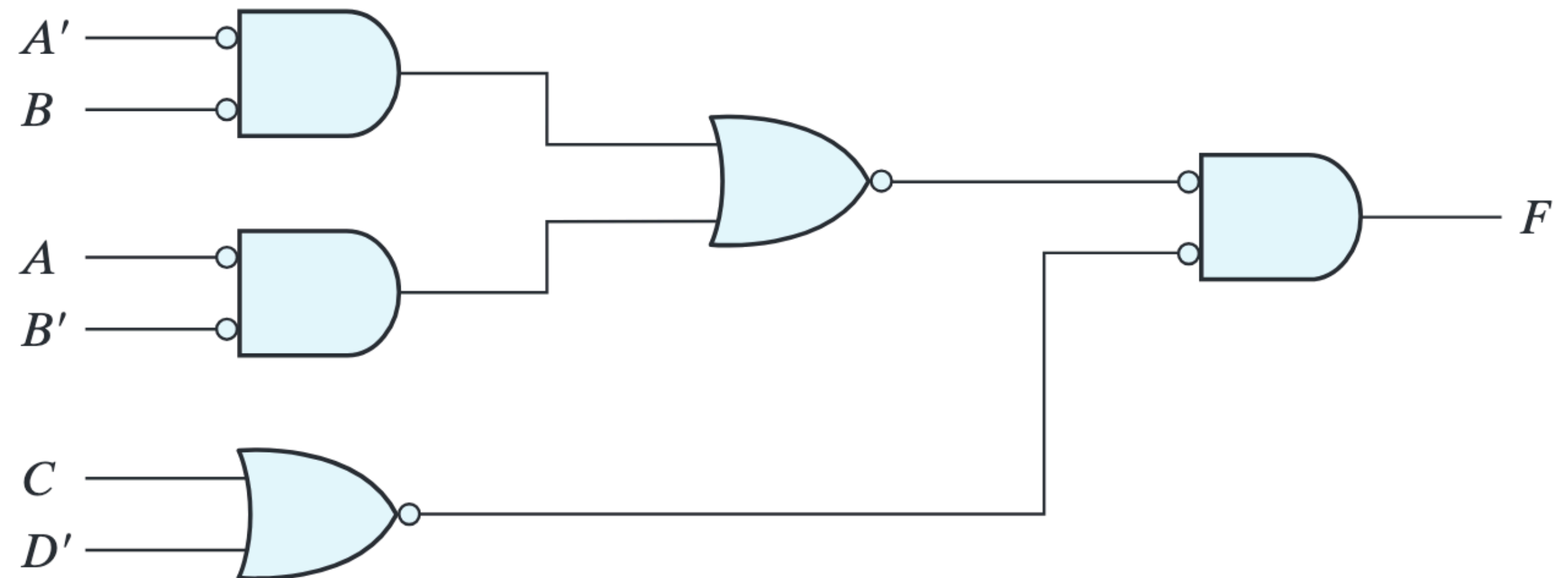
Notice that we need to invert E



Convert product-of-sums functions to NOR circuit

$$F = (AB' + A'B)(C + D')$$

- Need to invert A, B', A', B



Other two-level implementations

AND, OR, NAND, NOR

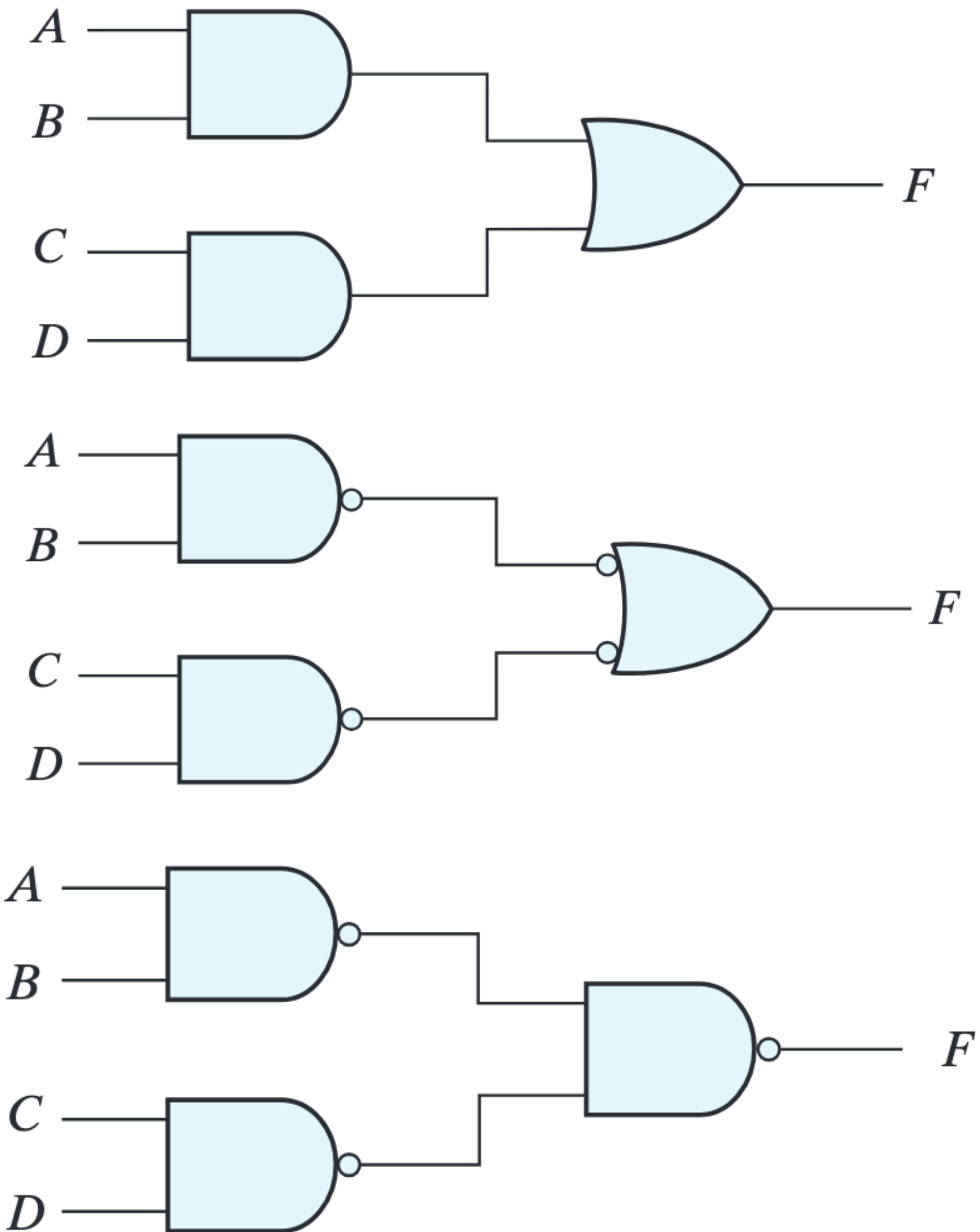
Degenerate to a single gate

- AND-AND: $(AB)(CD)$
- OR-OR: $(A+B)(C+D)$
- OR-NOR/NOR-OR: $((A+B)(C+D))'$
- AND-NAND/NAND-AND: $((AB)(CD))'$

Other two-level implementations

AND, OR, NAND, NOR

- AND-OR $\leftrightarrow$ NAND-NAND
 - Sum of products
- OR-AND $\leftrightarrow$ NOR-NOR
 - Product of sums
- NAND-AND $\leftrightarrow$ AND-NOR:
 - AND-INVERT-AND $\leftrightarrow$ AND-OR-INVERT
- NOR-OR $\leftrightarrow$ OR-NAND
 - OR-INVERT-OR $\leftrightarrow$ OR-AND-INVERT

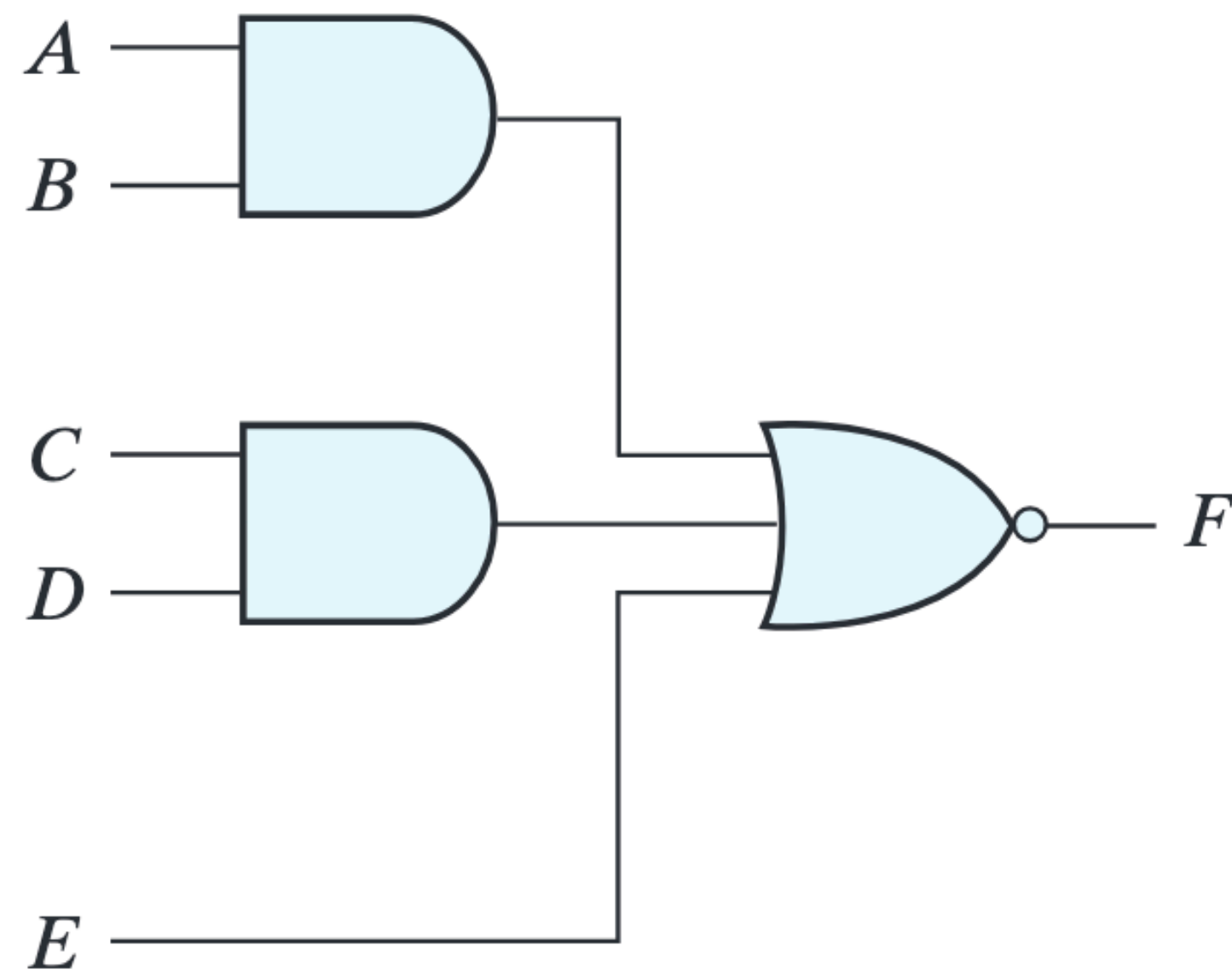


AND-OR-INVERT

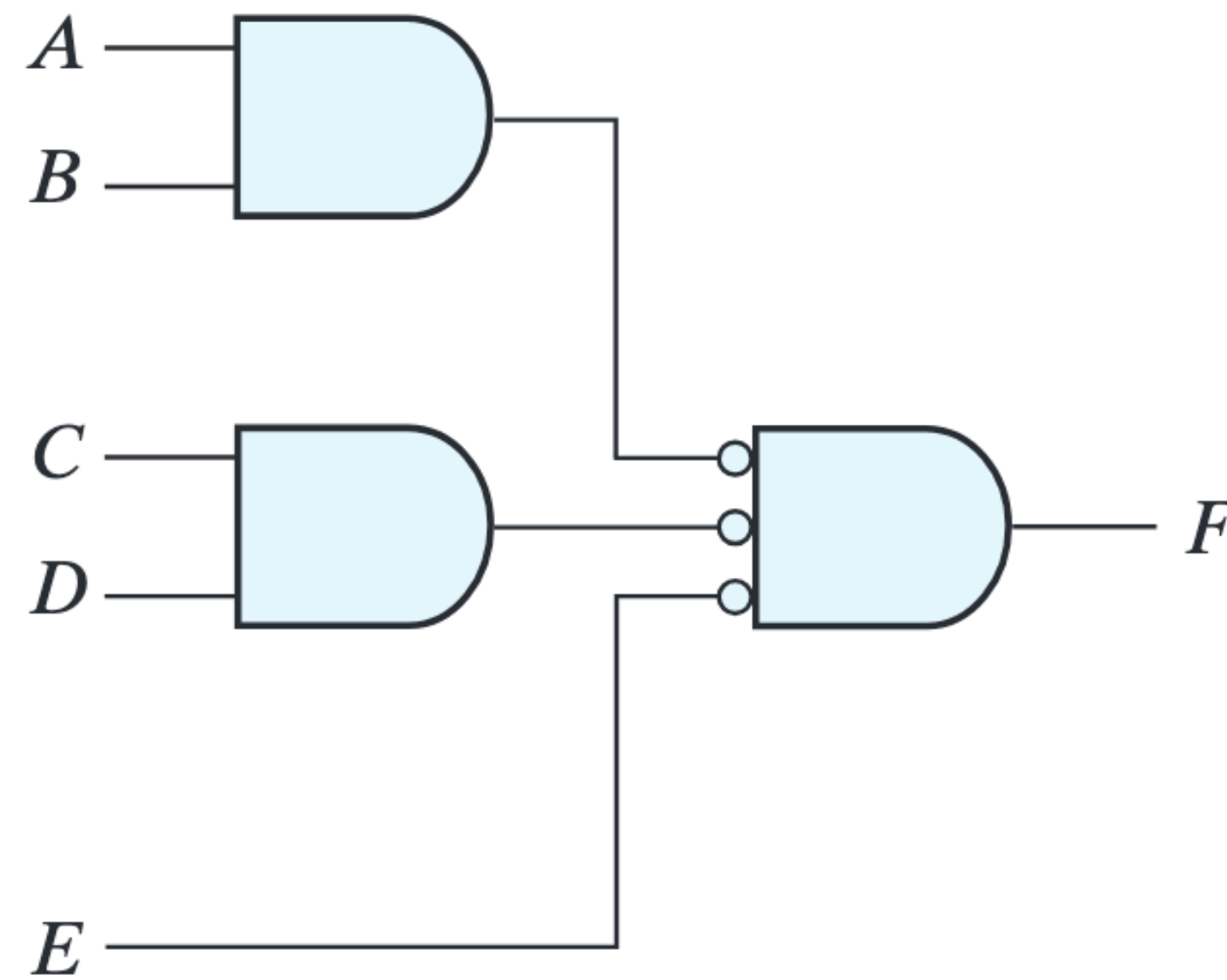
AND-NOR $\Leftrightarrow$ NAND-AND

- Push the bubble back

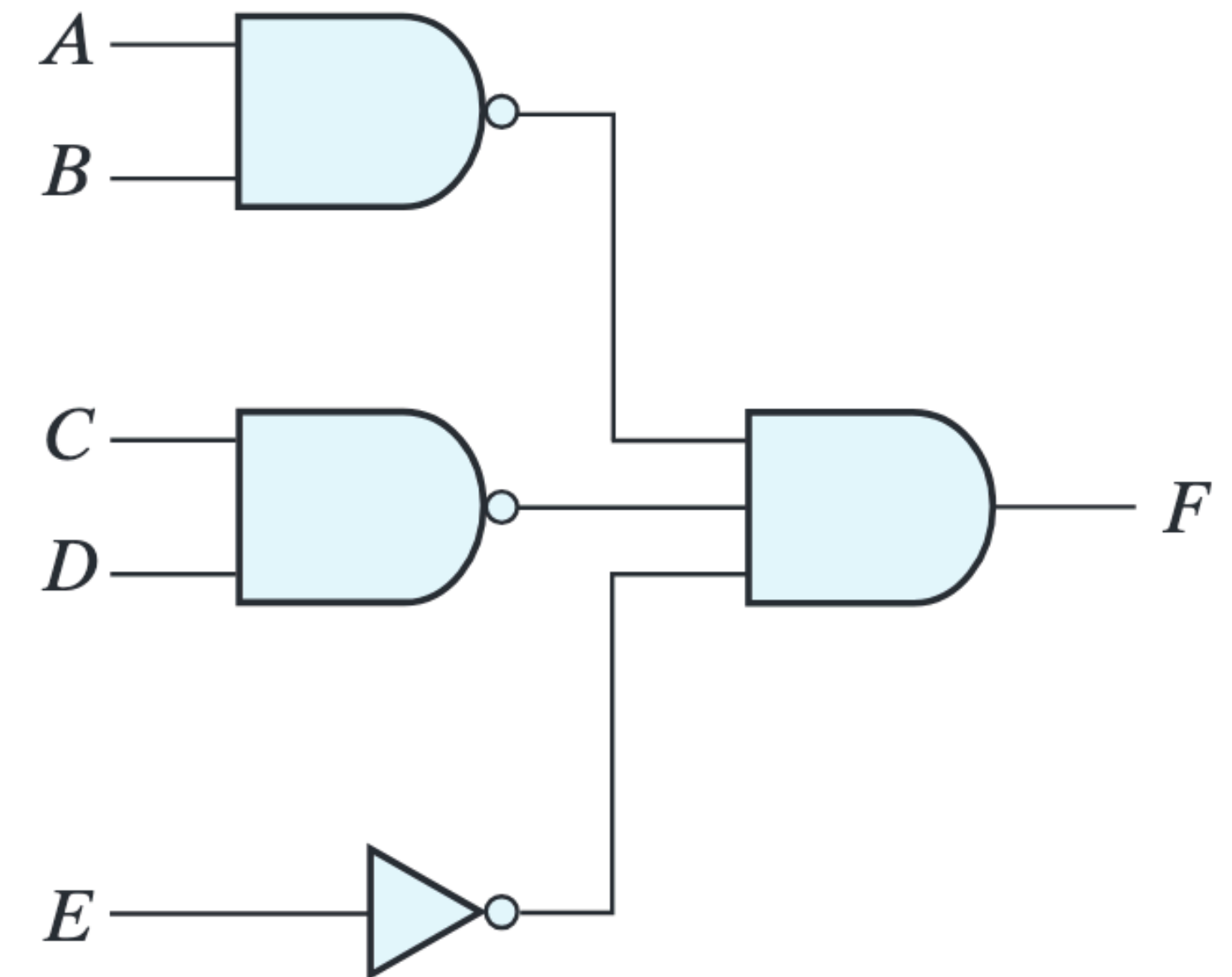
$$F = (AB + CD + E)'$$



(a) AND-NOR



(b) AND-NOR



(c) NAND-AND

Convert function to AND-OR-INVERT

AND-NOR <=> NAND-AND

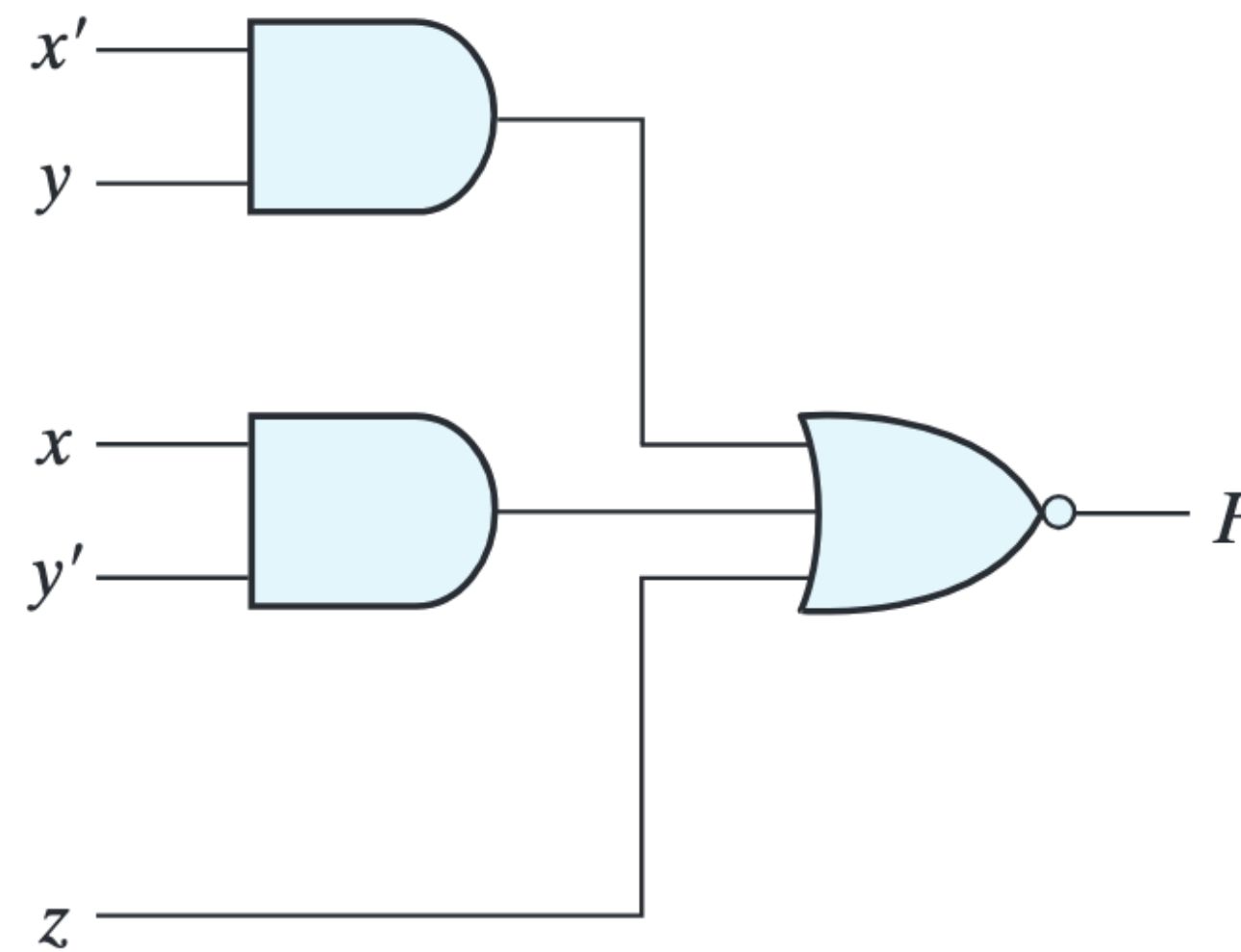
- Merge 0s in K-map to write F' as sum-of-products form.
- Draw AND-OR diagram for F' . Change OR to NOR to get F

$$F = \Sigma(0,6)$$

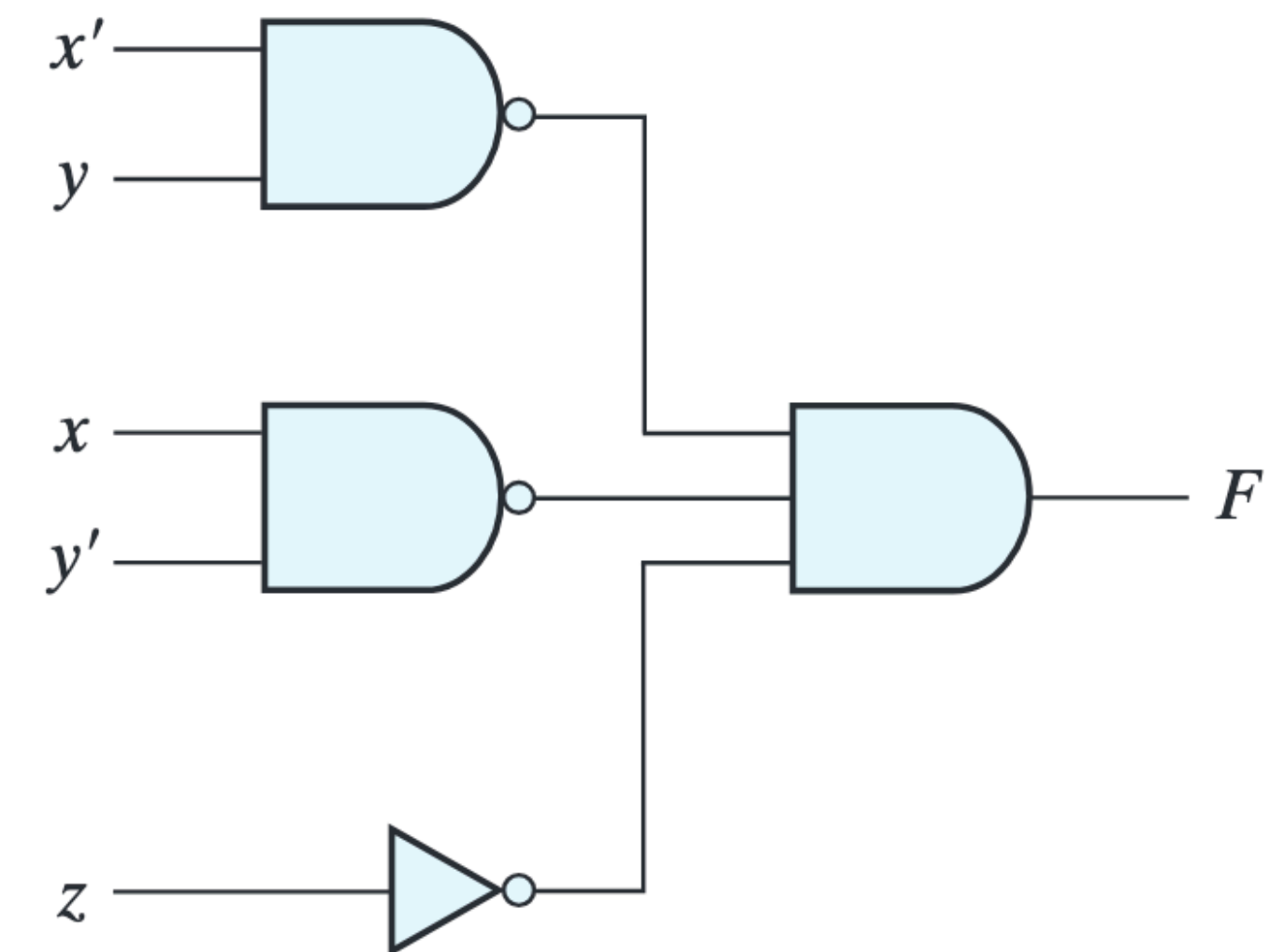
		y			
		z			
x	yz	00	01	11	10
		m_0	m_1	m_3	m_2
0	$x'y'z'$	1	0	0	0
1	x	m_4	m_5	m_7	m_6
		0	0	0	1

$$F = x'y'z' + xyz'$$

$$F' = x'y + xy' + z$$



AND-NOR



NAND-AND

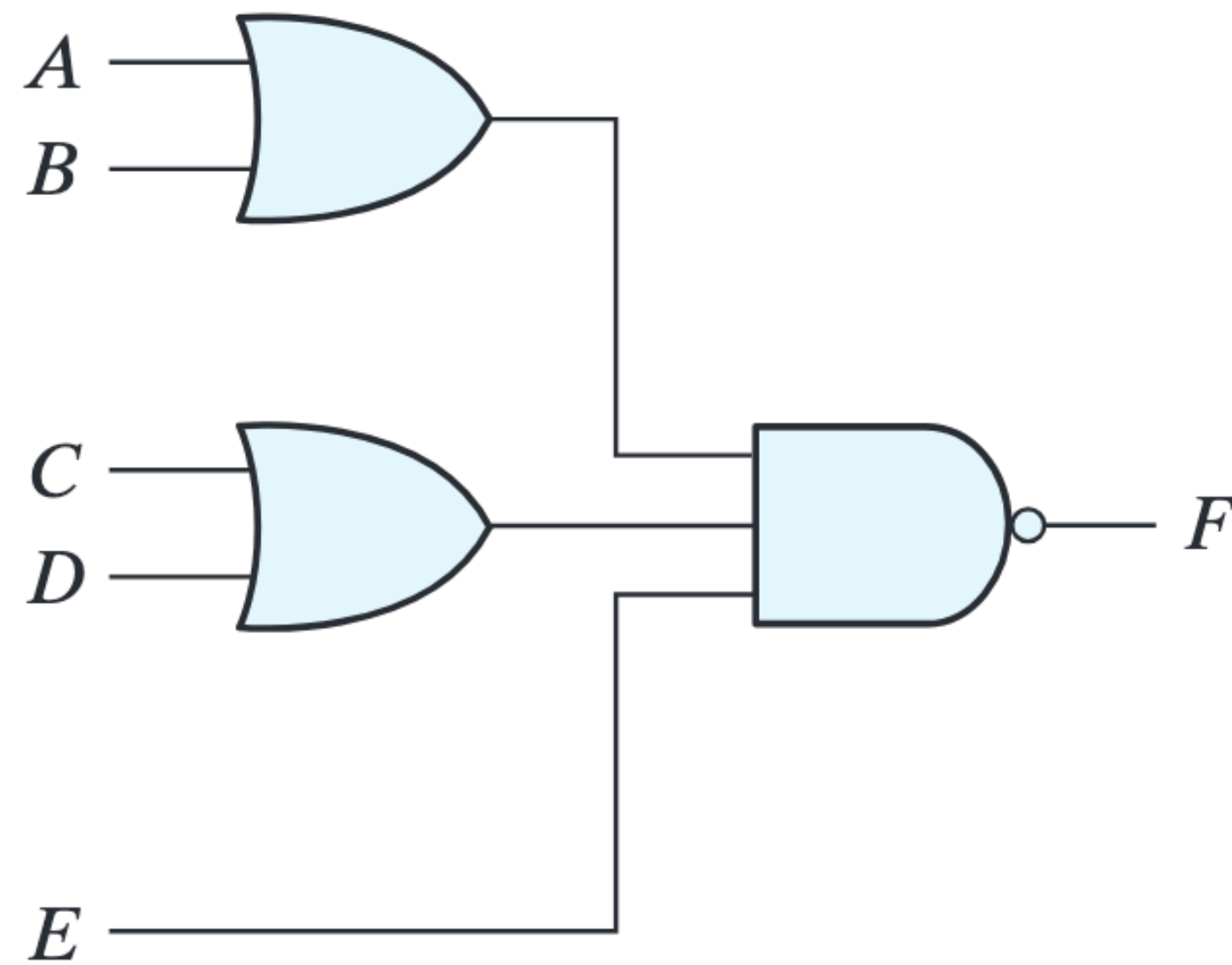
(b) $F = (x'y + xy' + z)'$

OR-AND-INVERT

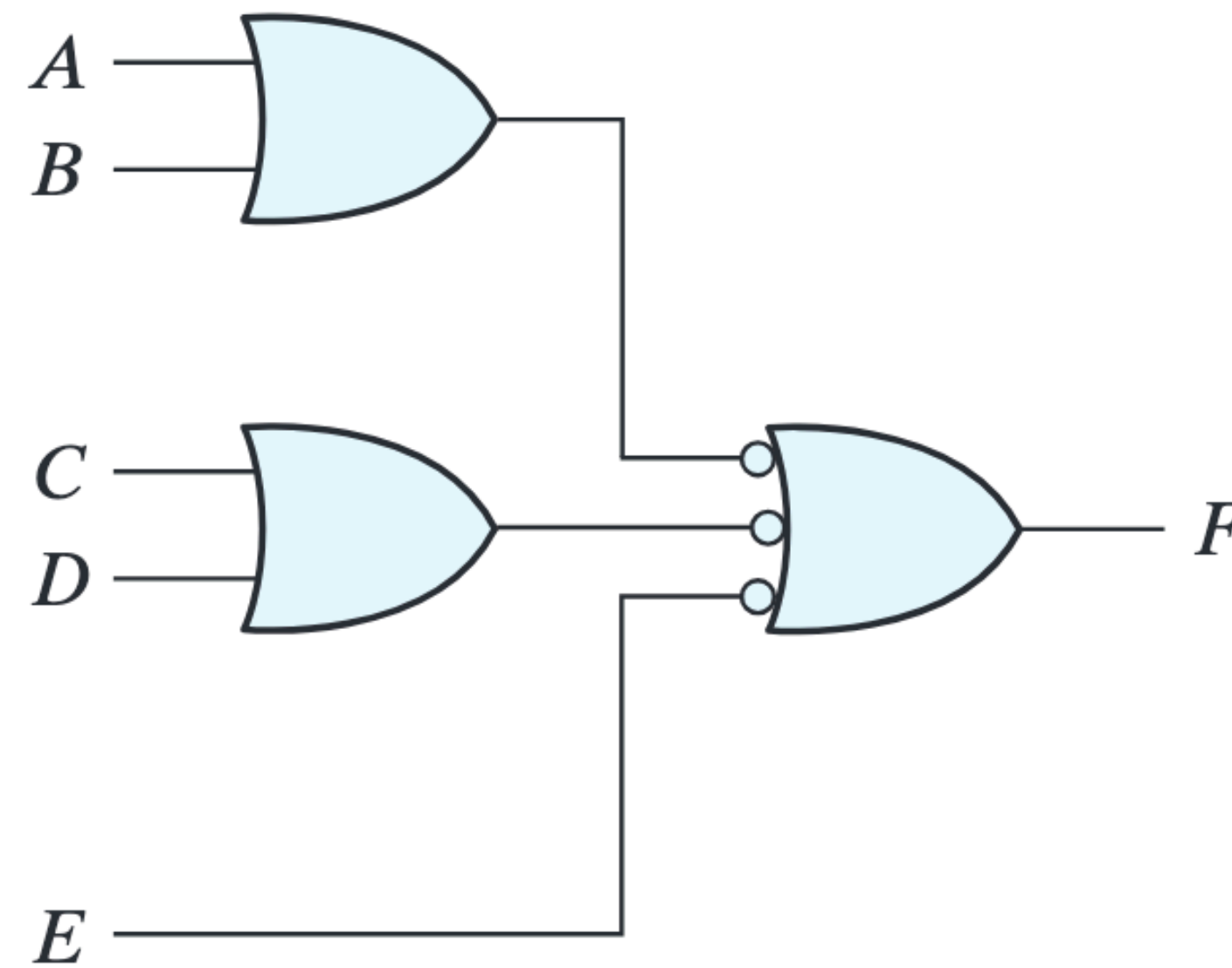
OR-NAND $\Leftrightarrow$ NOR-OR

- Push the bubble back

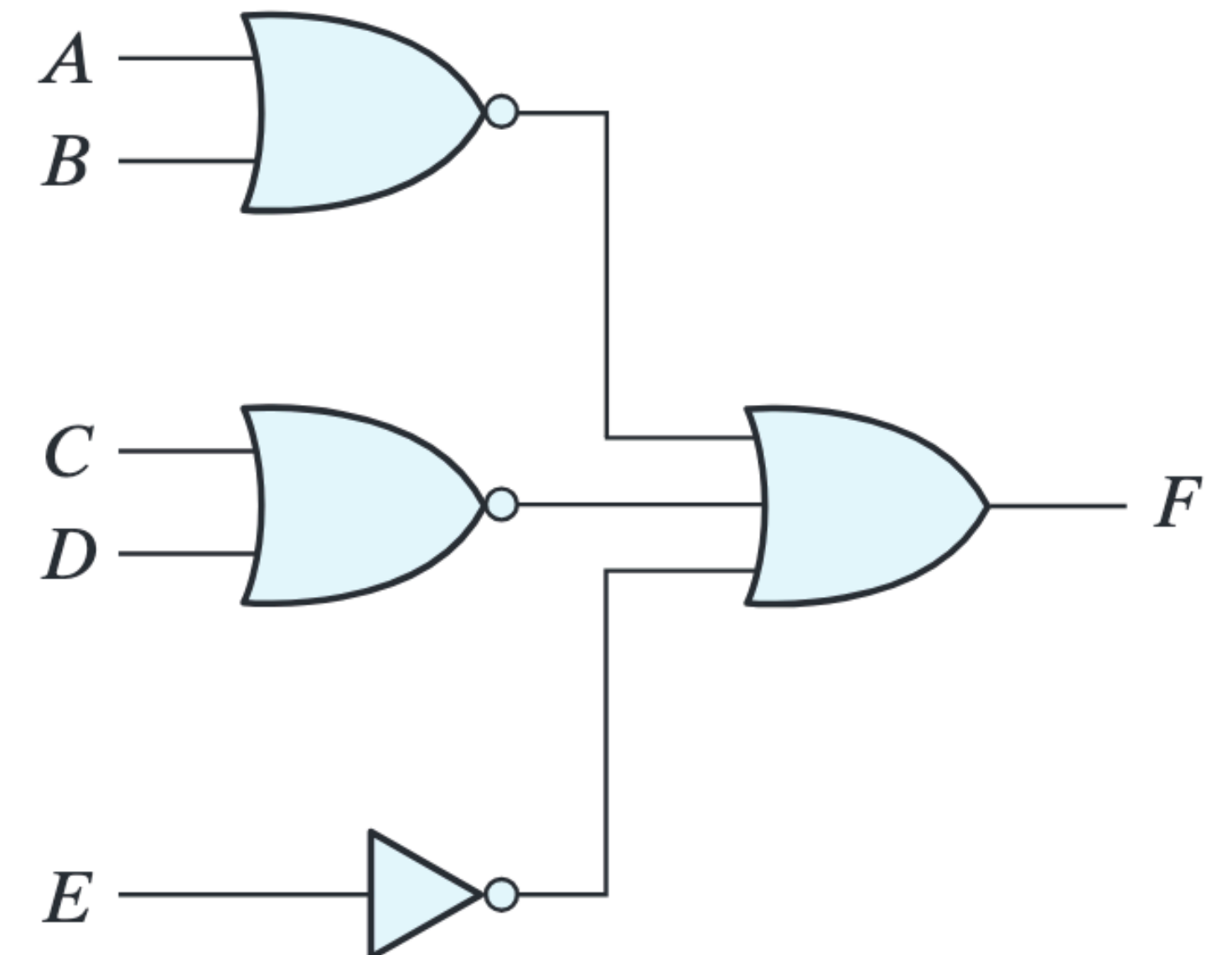
$$F = [(A + B)(C + D)E]'$$



(a) OR-NAND



(b) OR-NAND

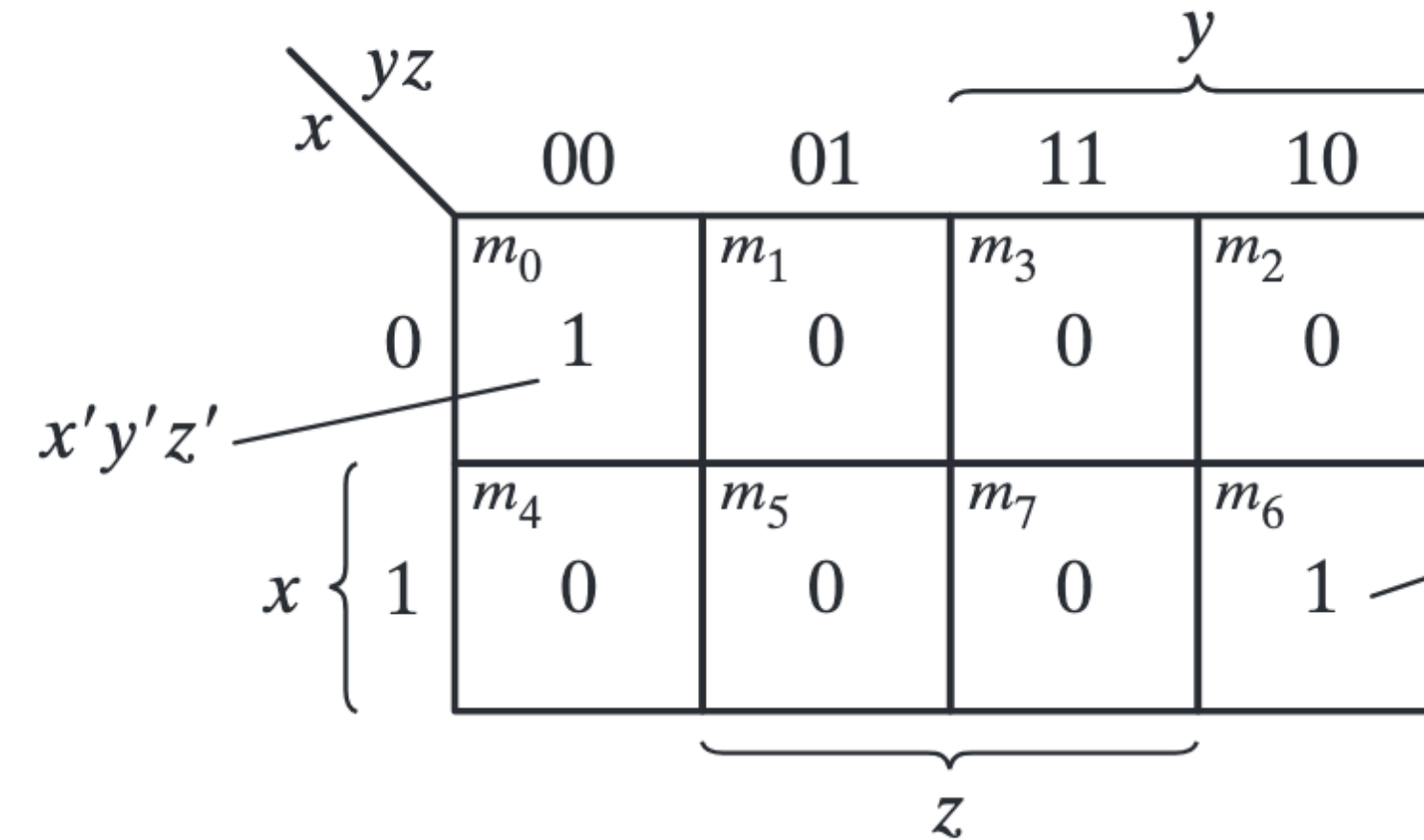


(c) NOR-OR

Convert function to OR-AND-INVERT

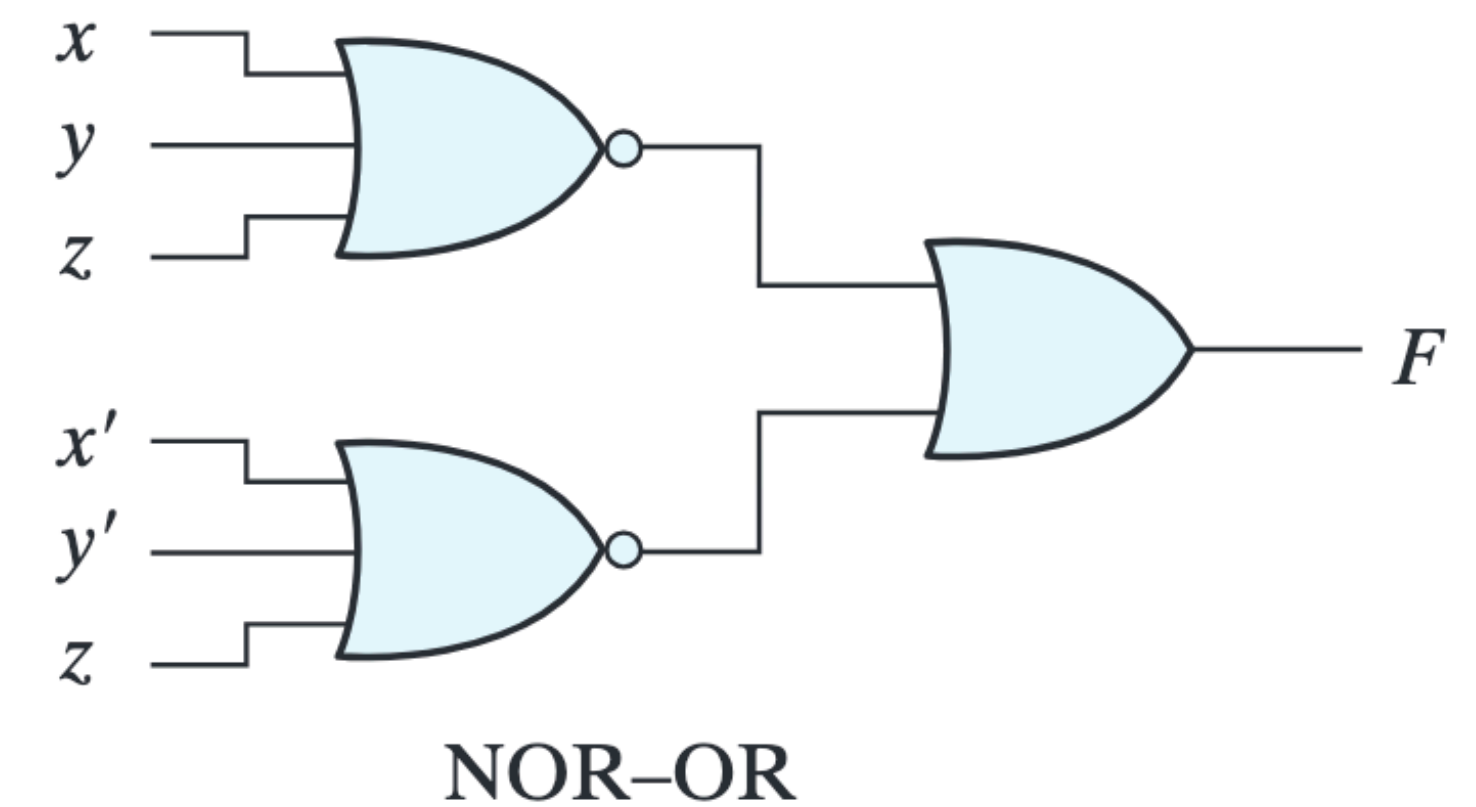
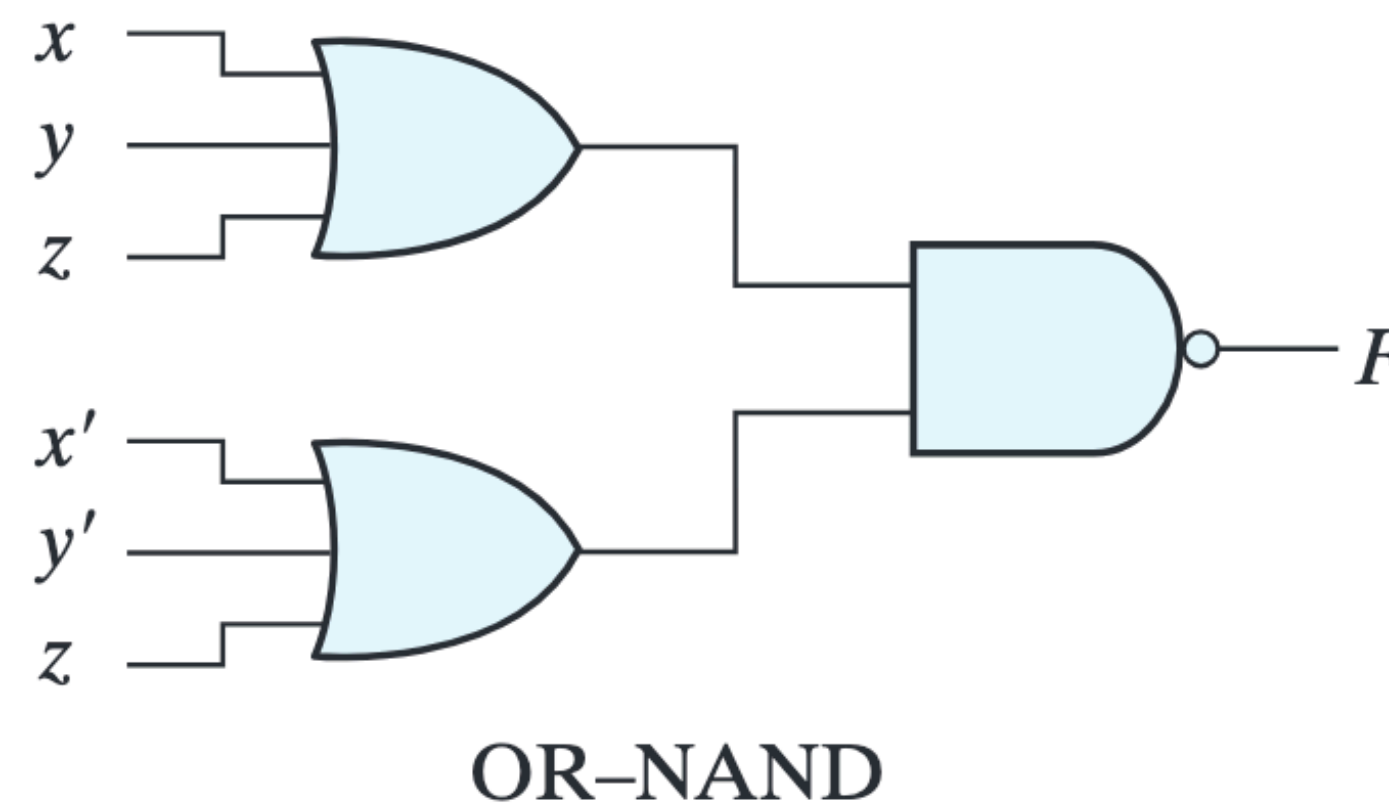
OR-NAND $\leftrightarrow$ NOR-OR

- Merge 1s in K-map to write F . Write F as sum-of-products form
- Take complement to get F' in product-of-sums form
- Draw OR-AND diagram for F' . Change AND to NAND to get F



$$F = x'y'z' + xyz'$$

$$F' = x'y + xy' + z$$



$$(c) F = [(x + y + z)(x' + y' + z)]'$$

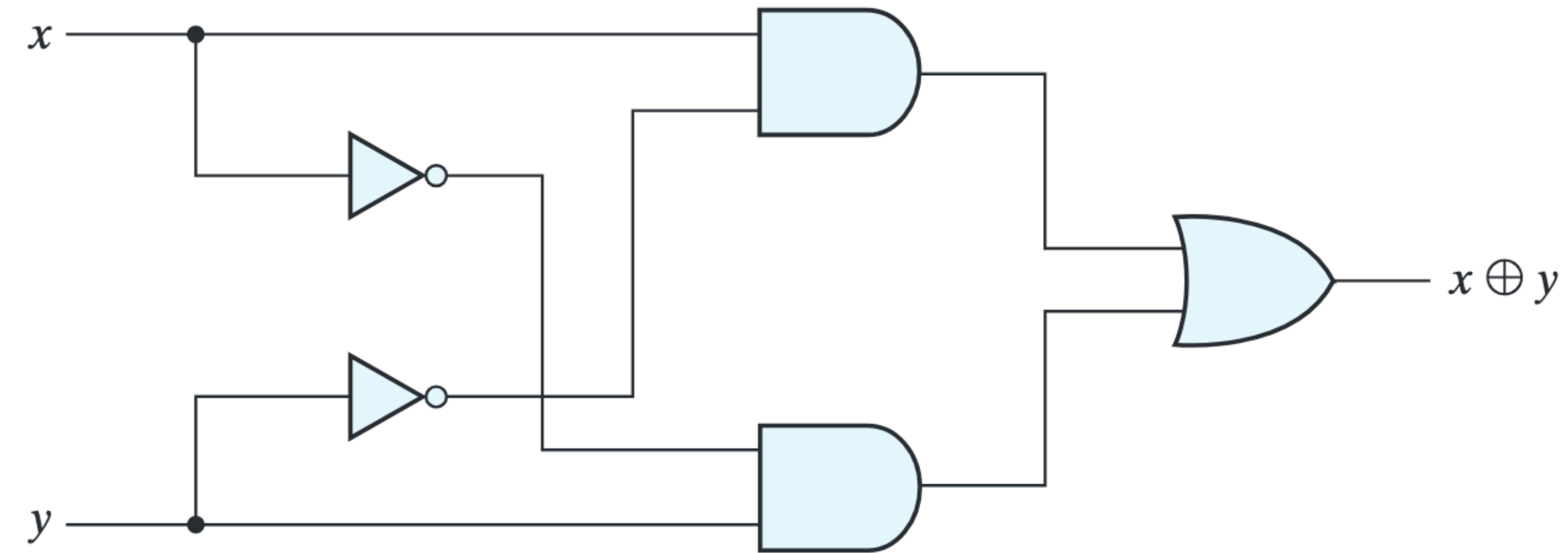
$$F = \Sigma(0,6)$$

XOR function

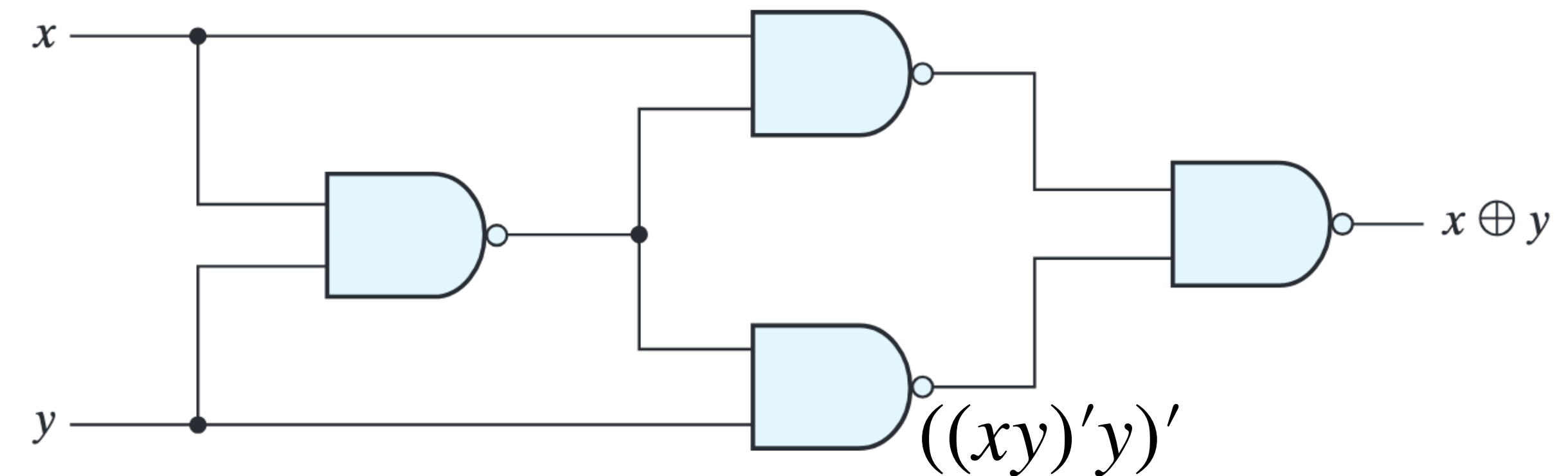
- $x \oplus y = xy' + x'y$. 1 when x and y differ. 0 when equal
- $(x \oplus y)' = (xy' + x'y)' = (x' + y)(x + y') = xy + x'y'$
 - 1 when x and y are equal. 0 when they differ.
- $x \oplus 0$
 - x
- $x \oplus 1$
 - x'
- $x \oplus x'$
 - 1
- $x \oplus x$
 - 0
- $x \oplus y'$
 - $x' \oplus y = (x \oplus y)'$
- $x \oplus y = y \oplus x$
- $(x \oplus y) \oplus z = x \oplus (y \oplus z)$

Implementing XOR

- $x \oplus y = xy' + x'y$
- Change AND to AND-INVERT and OR to INVERT-OR. Both are NAND
- Note that $(x'y)'$ is same as $((xy)'y)'$. Replace two NOT gates with one NAND gate



(a) Exclusive-OR with AND-OR-NOT gates



(b) Exclusive-OR with NAND gates